micarojas10 Fix typo in report title from 'EJERCICOS' to 'EJERCICIOS'

6abd72e · 2 minutes ago History

477 lines (320 loc) · 19.1 KB

Preview Code Blame Raw

Micaela Rojas - Especialización Análisis de Datos

Corrector: Marc Ponce Contenidos: SQL niveles 1,2 y 3

SPRINT 2 - SQL, INFORME DE EJERCICIOS

SPRINT 2 | SQLPresentación Este documento reúne los ejercicios prácticos realizados durante el Sprint 2 del curso de especialización en Data Analysis. El trabajo se organiza por niveles e incluye consultas SQL, operaciones sobre tablas, creación de vistas y tablas, relaciones entre datos y un modelo dimensional. Cada ejercicio incorpora una breve explicación y capturas de MySQL Workbench que muestran el desarrollo y los resultados obtenidos.

SQL – Nivel 1

En esta sección se recogen los ejercicios prácticos de SQL correspondientes al primer nivel de la formación en Data Analytics. Los ejercicios están organizados en dos partes, diferenciadas por los conjuntos de datos utilizados y los objetivos de trabajo.

Parte 1 – Ejercicios 1 al 7

Primer bloque de ejercicios prácticos de SQL, centrado en el trabajo con el primer conjunto de datos.

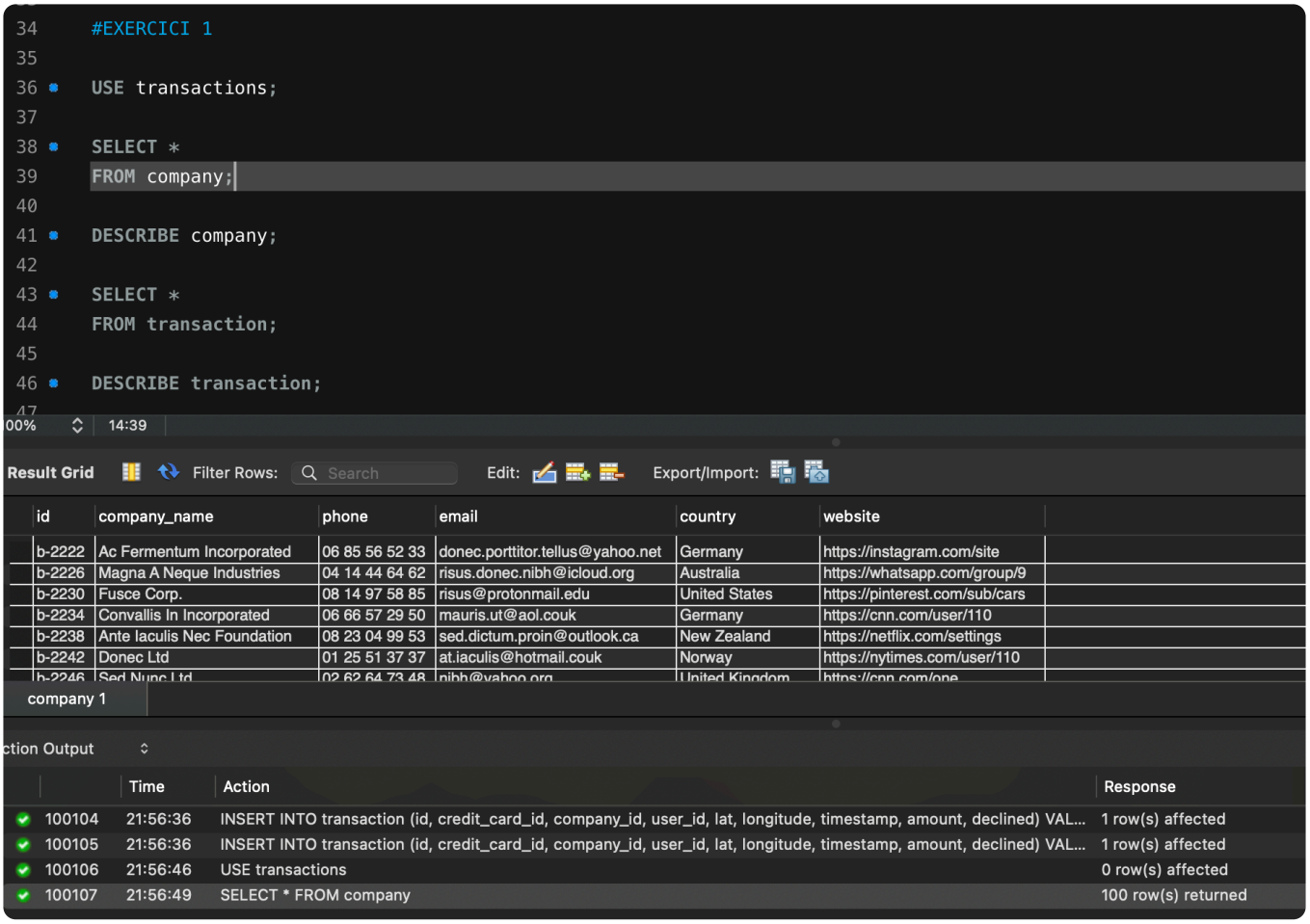
Ejercicio 1

Se realiza una primera exploración de las tablas company y transaction. Se consultan sus registros y se revisa la estructura de sus campos para reconocer la información disponible y cómo se relacionan las compras con las empresas. Las tablas company y transactions nos muestran un conjunto de datos sobre las compras de diferentes empresas en el mundo.

```sql
-- Creamos la tabla company
CREATE TABLE IF NOT EXISTS company (
    id VARCHAR(15) PRIMARY KEY,
    company_name VARCHAR(255),
    phone VARCHAR(15),
    email VARCHAR(100),
    country VARCHAR(100),
    website VARCHAR(255)
);

-- Creamos la tabla transaction
CREATE TABLE IF NOT EXISTS transaction (
    id VARCHAR(255) PRIMARY KEY,
    credit_card_id VARCHAR(15),
    company_id VARCHAR(20),
    user_id INT,
    lat FLOAT,
    longitude FLOAT,
    timestamp TIMESTAMP,
    amount DECIMAL(10, 2),
    declined BOOLEAN,
    FOREIGN KEY (company_id) REFERENCES company(id)
```

Las tablas company y transactions nos muestran un conjunto de datos sobre las compras de diferentes empresas en el mundo.



```
#EXERCICI 1

USE transactions;

SELECT *
FROM company;

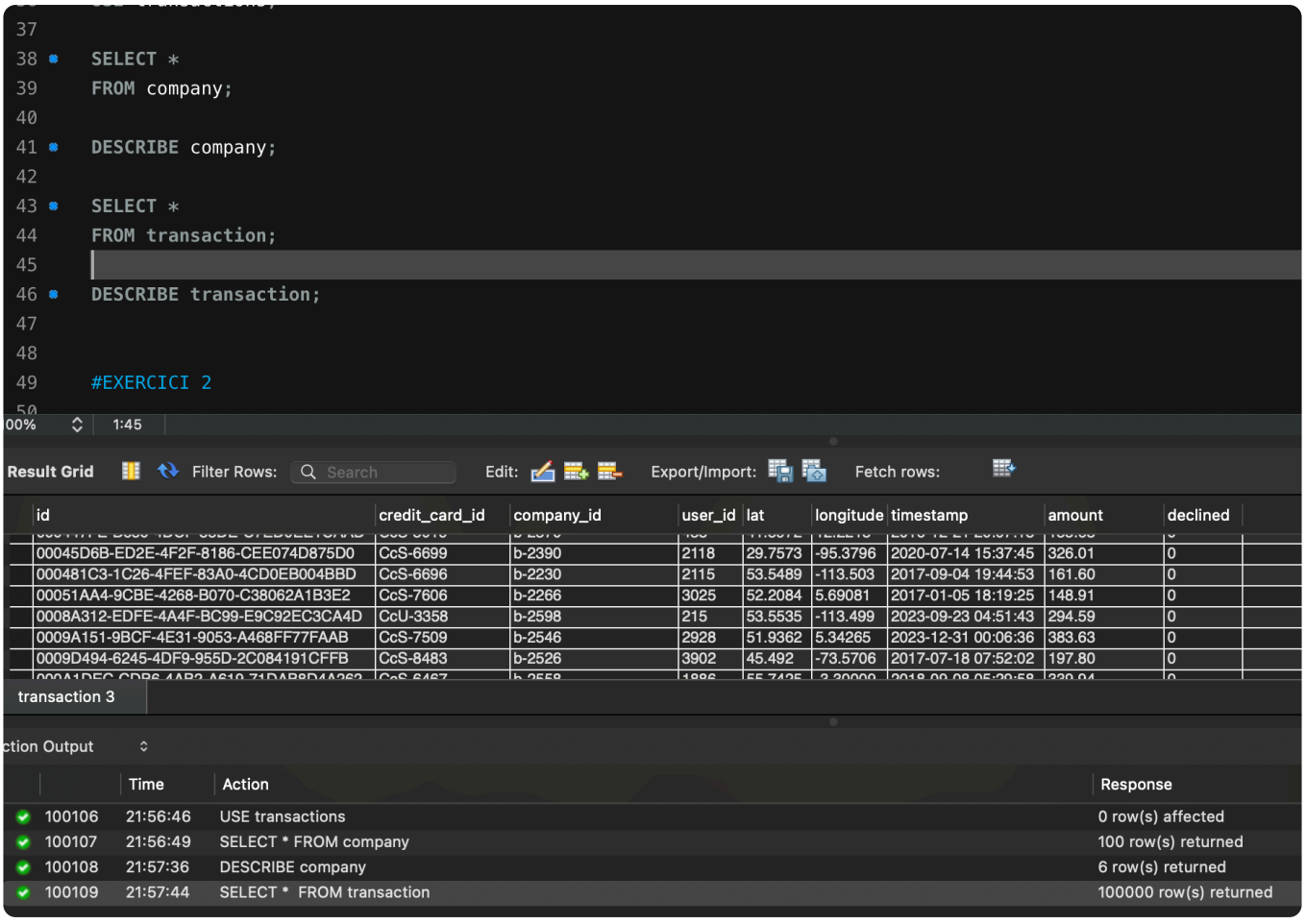
DESCRIBE company;

SELECT *
FROM transaction;

DESCRIBE transaction;
```

id	company_name	phone	email	country	website
b-2222	Ac Fermentum Incorporated	06 85 56 52 33	donec.porttitor.tellus@yahoo.net	Germany	https://instagram.com/site
b-2226	Magna A Neque Industries	04 14 44 64 62	risus.donec.nibh@icloud.org	Australia	https://whatsapp.com/group/9
b-2230	Fusce Corp.	08 14 97 58 85	risus@protonmail.edu	United States	https://pinterest.com/sub/cars
b-2234	Convallis In Incorporated	06 66 57 29 50	mauris.ut@aol.couk	Germany	https://cnn.com/user/110
b-2238	Ante Iaculis Nec Foundation	08 23 04 99 53	sed.dictum.proin@outlook.ca	New Zealand	https://netflix.com/settings
b-2242	Donec Ltd	01 25 51 37 37	at.iaculis@hotmail.couk	Norway	https://nytimes.com/user/110

	Time	Action	Response
100104	21:56:36	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VAL...	1 row(s) affected
100105	21:56:36	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VAL...	1 row(s) affected
100106	21:56:46	USE transactions	0 row(s) affected
100107	21:56:49	SELECT * FROM company	100 row(s) returned



```
SELECT *
FROM company;

DESCRIBE company;

SELECT *
FROM transaction;

DESCRIBE transaction;


#EXERCICI 2
```

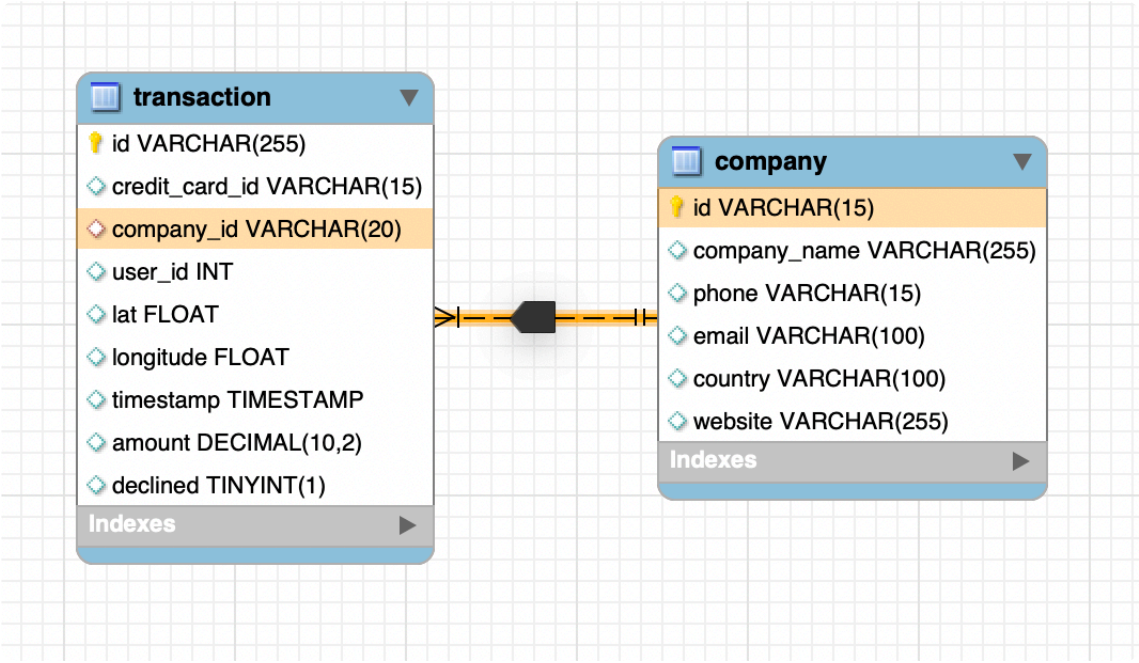
id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00045D6B-ED2E-4F2F-8186-CEE074D875D0	CcS-6699	b-2390	2118	29.7573	-95.3796	2020-07-14 15:37:45	326.01	0
000481C3-1C26-4FEF-83A0-4CD0EB004BBD	CcS-6696	b-2230	2115	53.5489	-113.503	2017-09-04 19:44:53	161.60	0
00051AA4-9CBE-4268-B070-C38062A1B3E2	CcS-7606	b-2266	3025	52.2084	5.69081	2017-01-05 18:19:25	148.91	0
0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	CcU-3358	b-2598	215	53.5535	-113.499	2023-09-23 04:51:43	294.59	0
0009A151-9BCF-4E31-9053-A468FF77FAAB	CcS-7509	b-2546	2928	51.9362	5.34265	2023-12-31 00:06:36	383.63	0
0009D494-6245-4DF9-955D-2C084191CFFB	CcS-8483	b-2526	3902	45.492	-73.5706	2017-07-18 07:52:02	197.80	0

	Time	Action	Response
100106	21:56:46	USE transactions	0 row(s) affected
100107	21:56:49	SELECT * FROM company	100 row(s) returned
100108	21:57:36	DESCRIBE company	6 row(s) returned
100109	21:57:44	SELECT * FROM transaction	100000 row(s) returned

La tabla company hace referencia a los datos de las diferentes empresas, en esta encontramos la siguiente relación con transaction, la cual hace referencia a las diferentes transacciones que ha tenido nuestra empresa de las empresas en nuestra

base de registros y los respectivos datos de las mismas en tiempo real y situación, *id* de Company es su PK, esta misma se usará como FK en la tabla de transactions.

A continuación, vemos el diagrama que demuestra la relación mencionada:



Ejercicio 2

Se relacionan las tablas company y transaction para identificar los países que generan ventas y calcular la media de importe por empresa. A partir de esos cálculos se identifica la compañía con la media de ventas más elevada.

```sql
## Llistat dels països que estan generant vendes.

SELECT DISTINCT country
FROM company
JOIN transaction
ON company.id = transaction.company_id
WHERE transaction.declined = 0
GROUP BY country;
```

00% 51:51

Result Grid Filter Rows: Search Export:

country
Netherlands
Sweden
Ireland
United States
Belgium
Canada
Germany
Norway
France
Italy
United Kingdom
New Zealand
China
Spain
Australia

Result 5

ction Output

	Time	Action	Response
100108	21:57:36	DESCRIBE company	6 row(s) returned
100109	21:57:44	SELECT * FROM transaction	100000 row(s) returned
100110	21:58:09	DESCRIBE transaction	9 row(s) returned
100111	22:07:00	SELECT DISTINCT country FROM company JOIN transaction ON company.id = transaction.company_id WHERE tran...	15 row(s) returned

```sql
## Des de quants països es generen les vendes.

SELECT COUNT(DISTINCT country) AS total_countries
FROM company
JOIN transaction
ON company.id = transaction.company_id
WHERE transaction.declined = 0;
```

100% 32:66

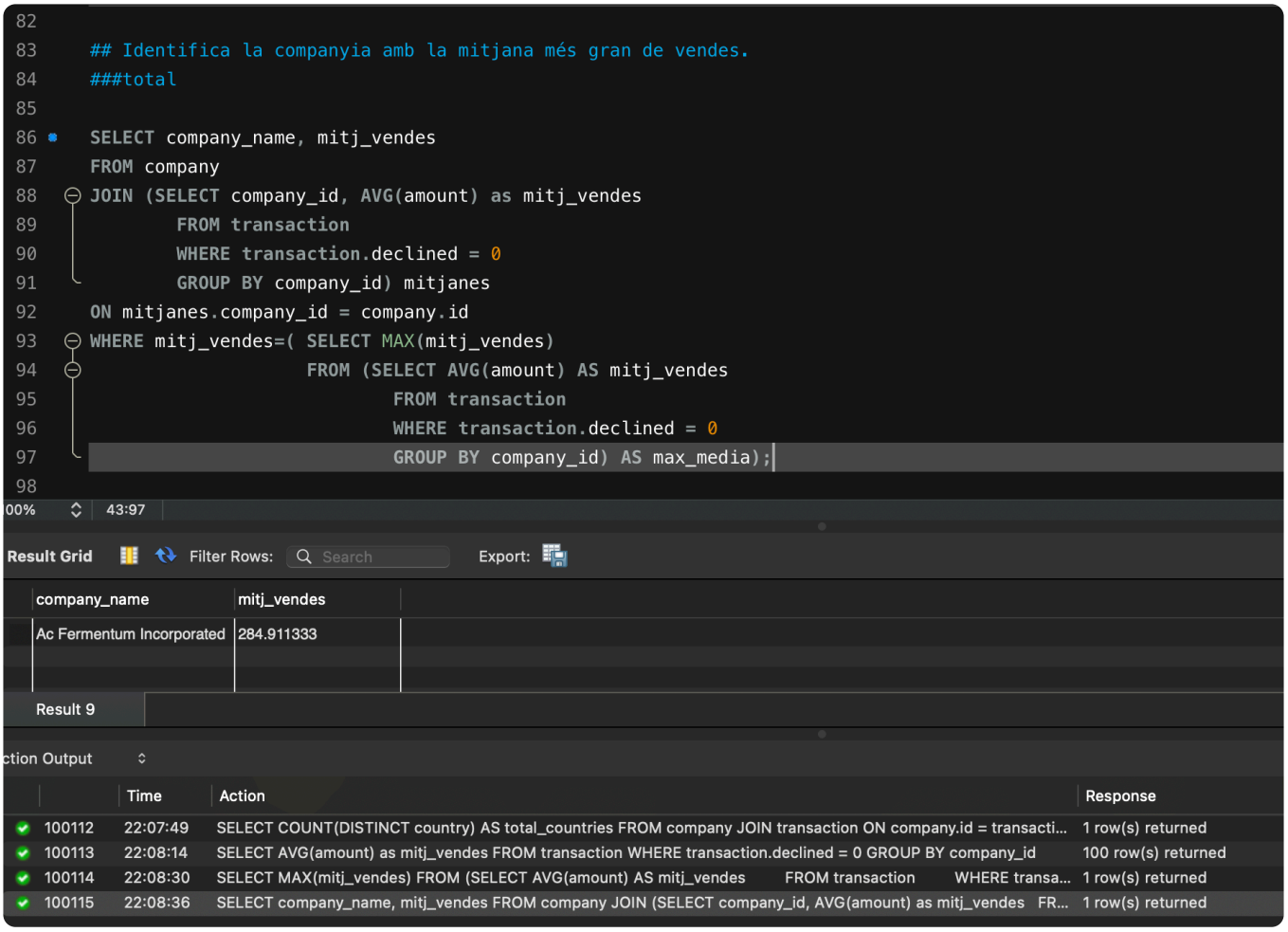
Result Grid Filter Rows: Search Export:

total_c...
15

Result 6

Action Output

	Time	Action	Response
100109	21:57:44	SELECT * FROM transaction	100000 row(s) returned
100110	21:58:09	DESCRIBE transaction	9 row(s) returned
100111	22:07:00	SELECT DISTINCT country FROM company JOIN transaction ON company.id = transaction.company_id WHERE tran...	15 row(s) returned
100112	22:07:49	SELECT COUNT(DISTINCT country) AS total_countries FROM company JOIN transaction ON company.id = transacti...	1 row(s) returned



Ejercicio 3

Se utilizan subconsultas para localizar empresas con transacciones cuyo importe supera la media de todas las transacciones. También se consulta si existen empresas sin transacciones registradas.

```sql
##Mostra totes les transaccions realitzades per empreses d'Alemanya.

### empreses alemanes
SELECT *
FROM company
WHERE country = 'germany';

###total
SELECT *
FROM transaction
WHERE EXISTS (
    SELECT id
    FROM company
    WHERE company.id = transaction.company_id
    AND country = 'Germany'
)
AND transaction.declined = 0;
```

100% 30:117

Result Grid Filter Rows: Search Edit: Export/Import: Fetch rows:

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00138D3B-206D-4C03-94B7-63A2676EB9B4	CcS-4899	b-2222	318	41.3781	12.447	2020-03-25 10:43:43	426.36	0
0013C1B6-3B84-4D6C-8154-E2B3FEBCA8E9	CcS-5070	b-2222	489	41.3814	2.18176	2020-12-17 18:15:37	316.90	0
00201A11-2E62-44C4-941D-198FC8DB77F0	CcU-3512	b-2222	193	55.5704	-3.65129	2021-01-22 23:44:27	453.04	0

transaction 11

ction Output

	Time	Action	Response
100114	22:08:30	SELECT MAX(mitj_vendes) FROM (SELECT AVG(amount) AS mitj_vendes FROM transaction WHERE transa...	1 row(s) returned
100115	22:08:36	SELECT company_name, mitj_vendes FROM company JOIN (SELECT company_id, AVG(amount) as mitj_vendes FR...	1 row(s) returned
100116	22:12:05	SELECT * FROM company WHERE country = 'germany'	8 row(s) returned
100117	22:12:14	SELECT * FROM transaction WHERE EXISTS (SELECT id FROM company WHERE company.id = transaction.c...	13269 row(s) returned

```sql
###amount superior a mitjana
SELECT company_id
FROM transaction
WHERE amount > (SELECT AVG(amount)
                FROM transaction);

##Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transacc
### total

SELECT *
FROM company
WHERE id IN (SELECT company_id
                FROM transaction
                WHERE amount > (SELECT AVG(amount)
                                FROM transaction));
```

100% 28:141

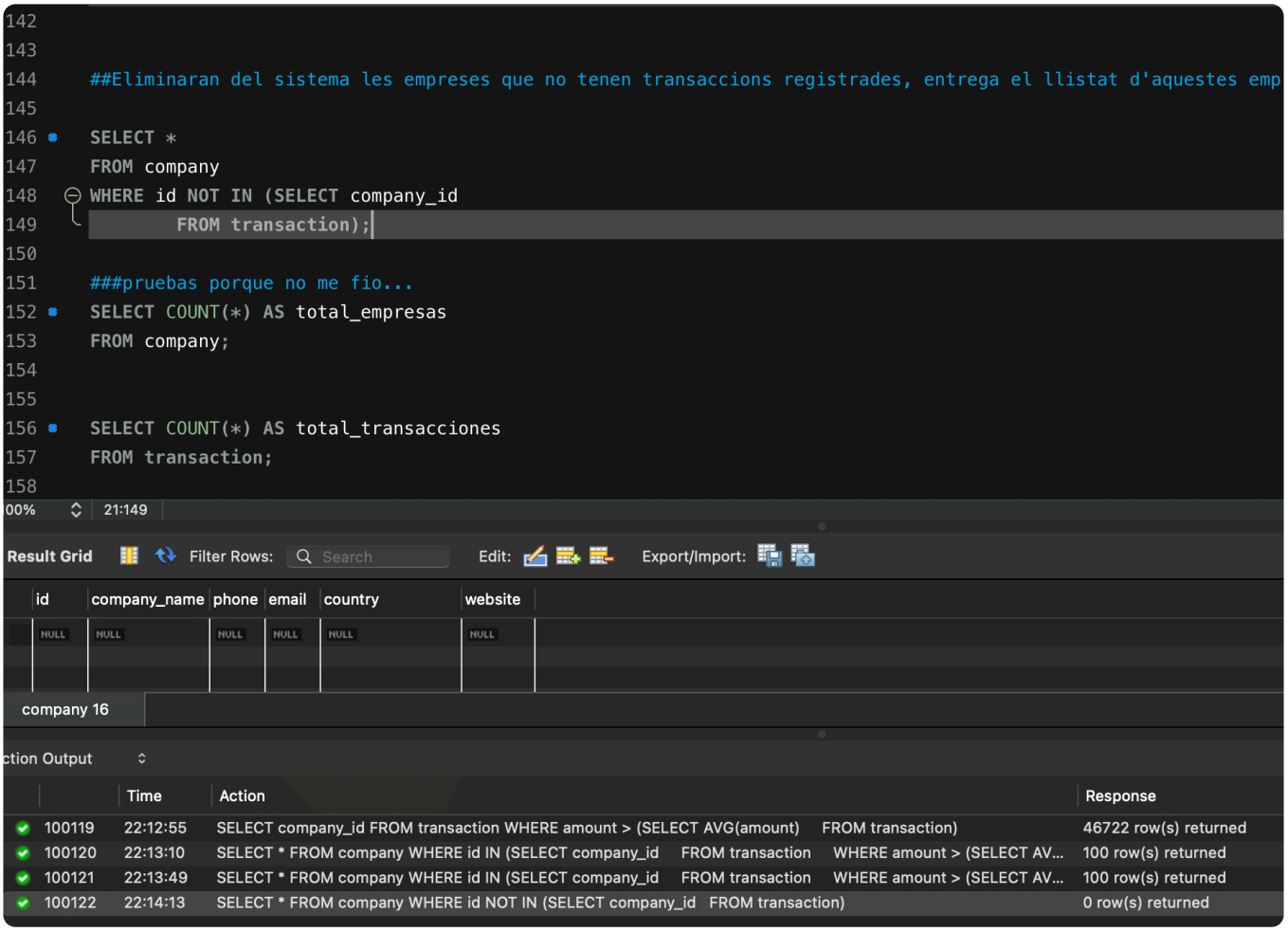
Result Grid Filter Rows: Search Edit: Export/Import:

id	company_name	phone	email	country	website
b-2230	Fusce Corp.	08 14 97 58 85	risus@protonmail.edu	United States	https://pinterest.com/sub/cars
b-2234	Convallis In Incorporated	06 66 57 29 50	mauris.ut@aol.couk	Germany	https://cnn.com/user/110
b-2238	Ante Iaculis Nec Foundation	08 23 04 99 53	sed.dictum.proin@outlook.ca	New Zealand	https://netflix.com/settings

company 15

ction Output

	Time	Action	Response
100118	22:12:50	SELECT AVG(amount) FROM transaction	1 row(s) returned
100119	22:12:55	SELECT company_id FROM transaction WHERE amount > (SELECT AVG(amount) FROM transaction)	46722 row(s) returned
100120	22:13:10	SELECT * FROM company WHERE id IN (SELECT company_id FROM transaction WHERE amount > (SELECT AV...	100 row(s) returned
100121	22:13:49	SELECT * FROM company WHERE id IN (SELECT company_id FROM transaction WHERE amount > (SELECT AV...	100 row(s) returned



Ejercicio 4

Se incorpora la tabla credit_card y se establece su relación con transaction mediante una clave foránea. Se revisa la estructura de la nueva tabla y se representa la relación entre las tablas en el diagrama.

```
CREATE TABLE IF NOT EXISTS credit_card (
    id VARCHAR(50) PRIMARY KEY,
    iban VARCHAR(50),
    pan VARCHAR(25),
    pin VARCHAR(10),
    cvv VARCHAR(10),
    expiring_date VARCHAR(10)
);
```

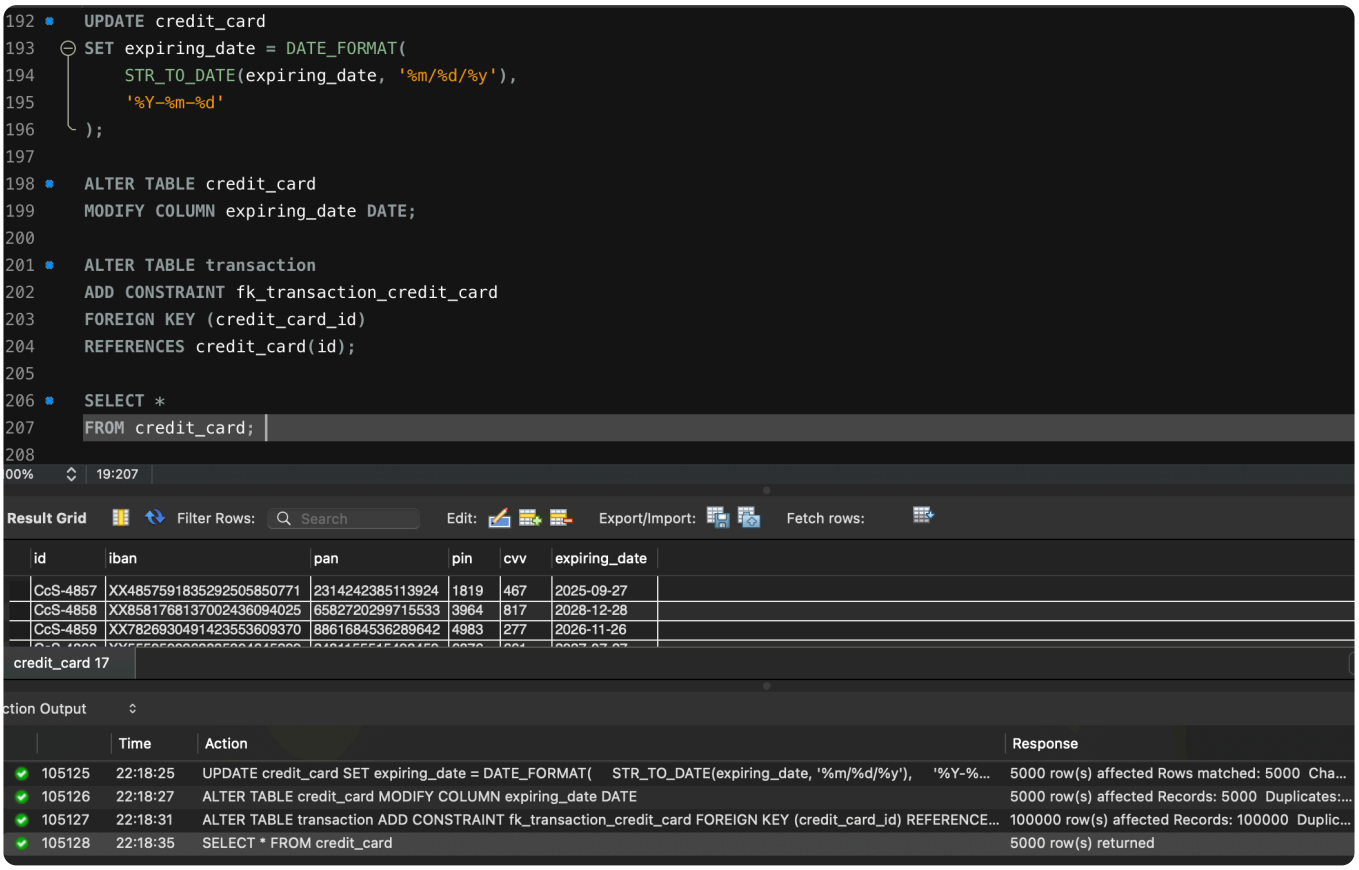
Al añadir la tabla credit card en la base de datos, ampliamos la información de las compañías y del método de pago de las transacciones.

```sql
SET SQL_SAFE_UPDATES = 0;

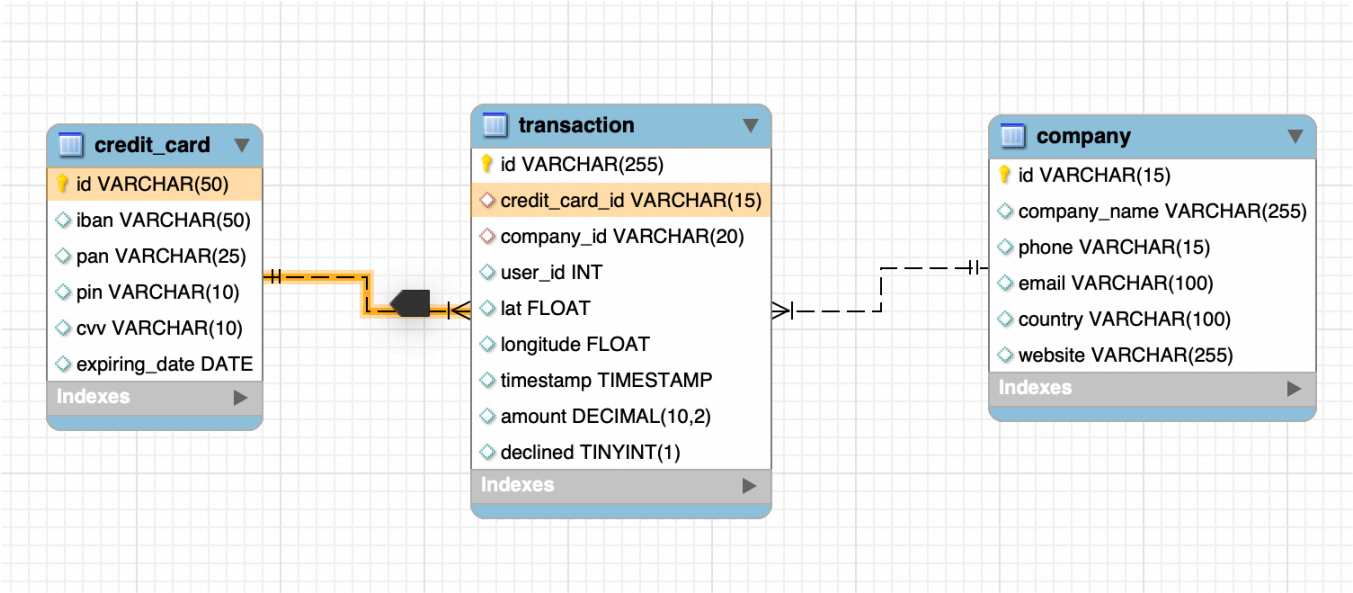
UPDATE credit_card
SET expiring_date = DATE_FORMAT(
    STR_TO_DATE(expiring_date, '%m/%d/%y'),
    '%Y-%m-%d'
);

ALTER TABLE credit_card
MODIFY COLUMN expiring_date DATE;

ALTER TABLE transaction
ADD CONSTRAINT fk_transaction_credit_card
FOREIGN KEY (credit_card_id)
REFERENCES credit_card(id);
```

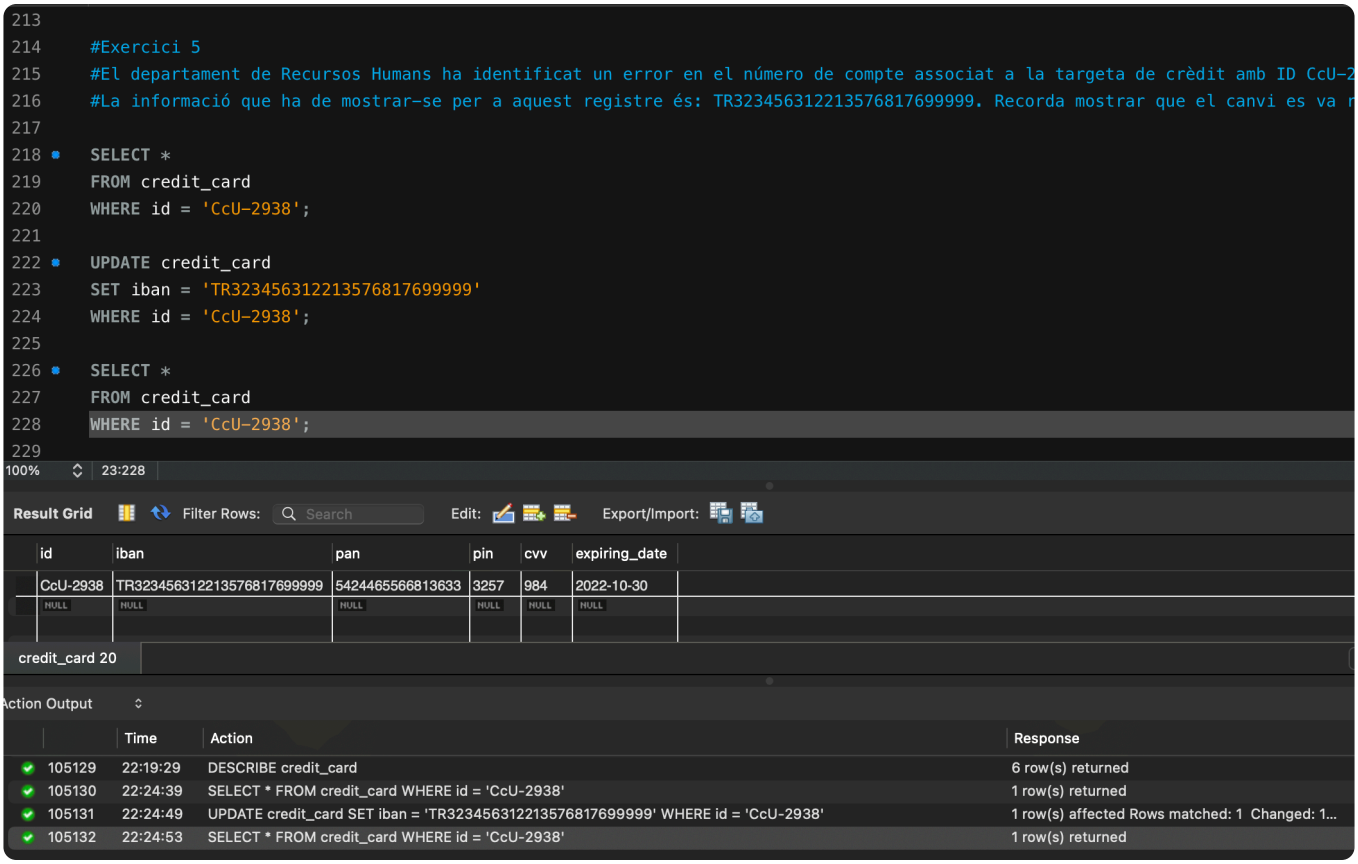


Los datos de esta nueva tabla nos muestran los diferentes datos de cada tarjeta de crédito utilizada para las transferencias en transactions que usan las diferentes empresas. Esta se relaciona con el *card_id* que se utiliza en transacciones, siendo asi la FK en ella y la PK en *credit_card*. Esto formara un modelo de estrella, siendo la central la tabla *transactions*.



Ejercicio 5

Se actualiza el IBAN de una tarjeta concreta mediante UPDATE, utilizando el identificador de la tarjeta en la condición WHERE. Después se consulta el registro para comprobar el cambio.



```
213
214    #Exercici 5
215    #El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2
216    #La informació que ha de mostrar-se per a aquest registre és: TR323456312213576817699999. Recorda mostrar que el canvi es va r
217
218    SELECT *
219    FROM credit_card
220    WHERE id = 'CcU-2938';
221
222    UPDATE credit_card
223    SET iban = 'TR323456312213576817699999'
224    WHERE id = 'CcU-2938';
225
226    SELECT *
227    FROM credit_card
228    WHERE id = 'CcU-2938';
229
```

100% 23:228

Result Grid Filter Rows: Search Edit: Export/Import:

id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	2022-10-30
NULL	NULL	NULL	NULL	NULL	NULL

credit_card 20

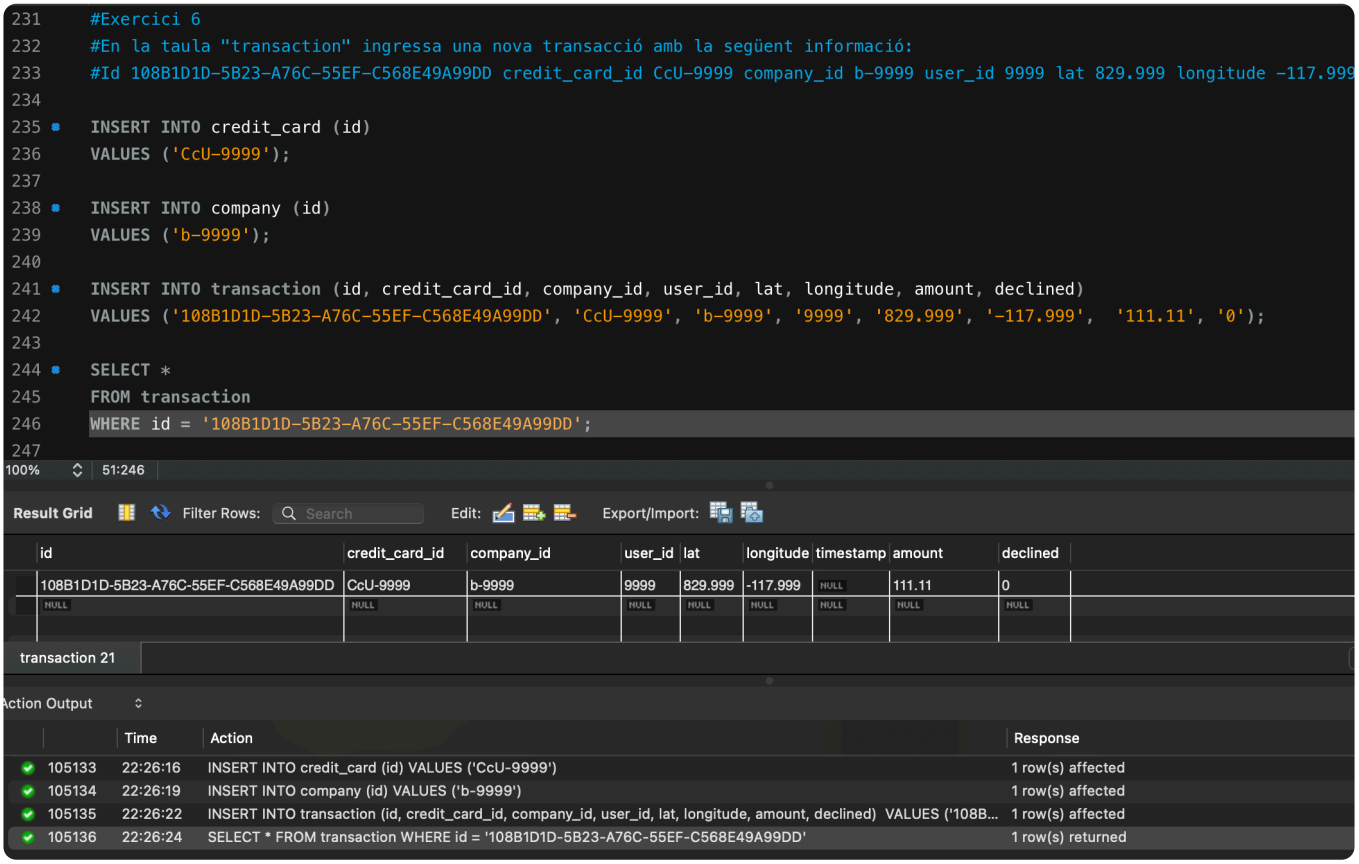
Action Output

	Time	Action	Response
105129	22:19:29	DESCRIBE credit_card	6 row(s) returned
105130	22:24:39	SELECT * FROM credit_card WHERE id = 'CcU-2938'	1 row(s) returned
105131	22:24:49	UPDATE credit_card SET iban = 'TR323456312213576817699999' WHERE id = 'CcU-2938'	1 row(s) affected Rows matched: 1 Changed: 1...
105132	22:24:53	SELECT * FROM credit_card WHERE id = 'CcU-2938'	1 row(s) returned

Ejercicio 6

Se insertan nuevos registros en las tablas credit_card, company y transaction. El ejercicio permite practicar INSERT y comprobar que los registros relacionados se

incorporan correctamente.



```
#Exercici 6
#En la taula "transaction" ingressa una nova transacció amb la següent informació:
#Id 108B1D1D-5B23-A76C-55EF-C568E49A99DD credit_card_id CcU-9999 company_id b-9999 user_id 9999 lat 829.999 longitude -117.999

INSERT INTO credit_card (id)
VALUES ('CcU-9999');

INSERT INTO company (id)
VALUES ('b-9999');

INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
VALUES ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', '9999', '829.999', '-117.999',  '111.11', '0');

SELECT *
FROM transaction
WHERE id = '108B1D1D-5B23-A76C-55EF-C568E49A99DD';
```

100% 51:246

Result Grid Filter Rows: Search Edit: Export/Import:

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
108B1D1D-5B23-A76C-55EF-C568E49A99DD	CcU-9999	b-9999	9999	829.999	-117.999	NULL	111.11	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

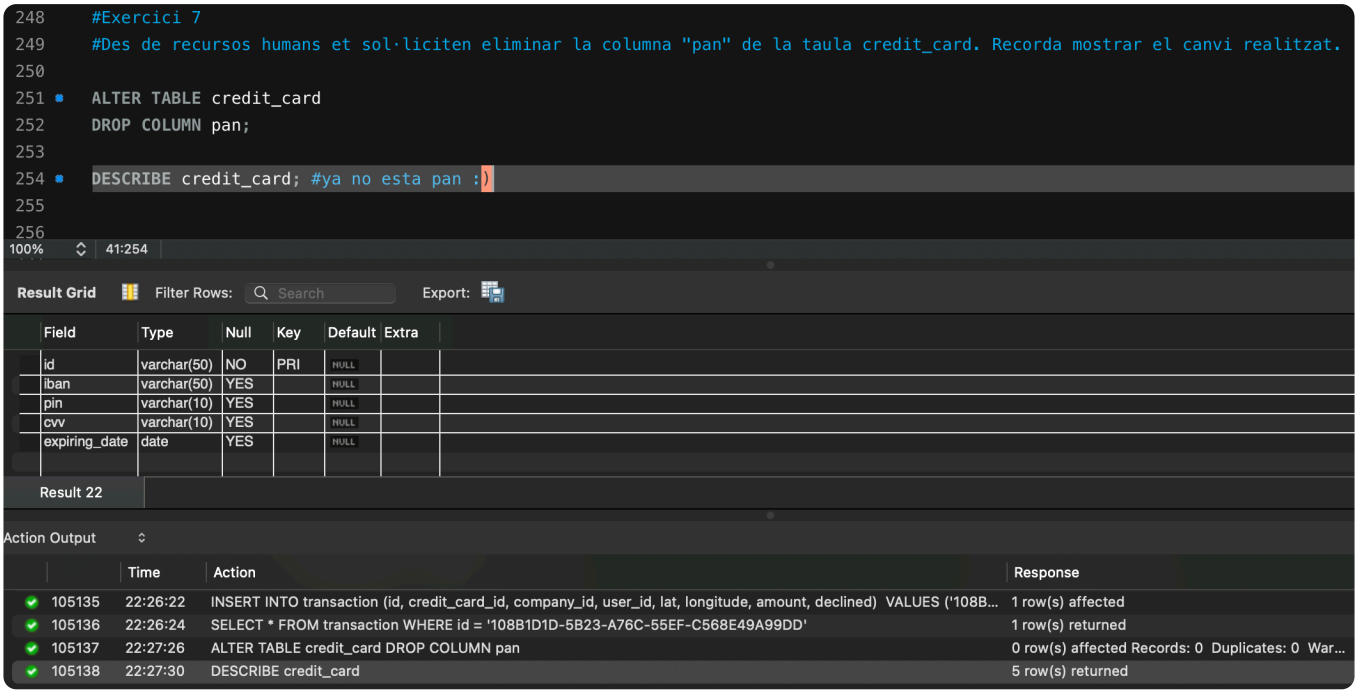
transaction 21

Action Output

	Time	Action	Response
105133	22:26:16	INSERT INTO credit_card (id) VALUES ('CcU-9999')	1 row(s) affected
105134	22:26:19	INSERT INTO company (id) VALUES ('b-9999')	1 row(s) affected
105135	22:26:22	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VALUES ('108B...	1 row(s) affected
105136	22:26:24	SELECT * FROM transaction WHERE id = '108B1D1D-5B23-A76C-55EF-C568E49A99DD'	1 row(s) returned

Ejercicio 7

Se modifica la estructura de credit_card eliminando la columna pan mediante ALTER TABLE y DROP COLUMN. Finalmente, se comprueba la estructura actualizada de la tabla.



```
#Exercici 7
#Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recorda mostrar el canvi realitzat.

ALTER TABLE credit_card
DROP COLUMN pan;

DESCRIBE credit_card; #ya no esta pan :)
```

100% 41:254

Result Grid Filter Rows: Search Export:

Field	Type	Null	Key	Default	Extra
id	varchar(50)	NO	PRI	NULL	
iban	varchar(50)	YES		NULL	
pin	varchar(10)	YES		NULL	
cvv	varchar(10)	YES		NULL	
expiring_date	date	YES		NULL	

Result 22

Action Output

	Time	Action	Response
105135	22:26:22	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VALUES ('108B...	1 row(s) affected
105136	22:26:24	SELECT * FROM transaction WHERE id = '108B1D1D-5B23-A76C-55EF-C568E49A99DD'	1 row(s) returned
105137	22:27:26	ALTER TABLE credit_card DROP COLUMN pan	0 row(s) affected Records: 0 Duplicates: 0 War...
105138	22:27:30	DESCRIBE credit_card	5 row(s) returned

Parte 2 – Ejercicios 8 al 10

Segundo bloque de ejercicios, desarrollado a partir de un conjunto de datos diferente.

En esta parte se trabaja con la creación e importación de tablas, la transformación de datos y el diseño de un modelo dimensional mediante un esquema en estrella (*Star Schema*).

Ejercicio 8

Se construye un modelo dimensional en estrella a partir de los archivos CSV. Se crean e importan las tablas de dimensiones y la tabla de hechos, se transforman los campos necesarios y se definen las relaciones entre ellas.

Comenzamos creando la nueva database y la primera tabla de datos de usuarios *dim_user*:

```
256
257    #Exercici 8
258    #Estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitz
259
260    CREATE DATABASE transactions_star;
261
262    USE transactions_star;
263
264    CREATE TABLE dim_user (
265        id INT PRIMARY KEY,
266        name VARCHAR(100),
267        surname VARCHAR(100),
268        phone VARCHAR(30),
269        email VARCHAR(150),
270        birth_date DATE,
271        country VARCHAR(100),
272        city VARCHAR(100),
273        postal_code VARCHAR(20),
274        address VARCHAR(255),
275        signup_date DATE,
276        user_segment VARCHAR(50),
277        income_band VARCHAR(30)
278    );
279
```

100% 3:278

Action Output

	Time	Action	Response
105138	22:27:30	DESCRIBE credit_card	5 row(s) returned
105139	22:28:22	CREATE DATABASE transactions_star	1 row(s) affected
105140	22:28:29	USE transactions_star	0 row(s) affected
105141	22:30:18	CREATE TABLE dim_user (id INT PRIMARY KEY, name VARCHAR(100), surname VARCHAR(100), phone VA...	0 row(s) affected

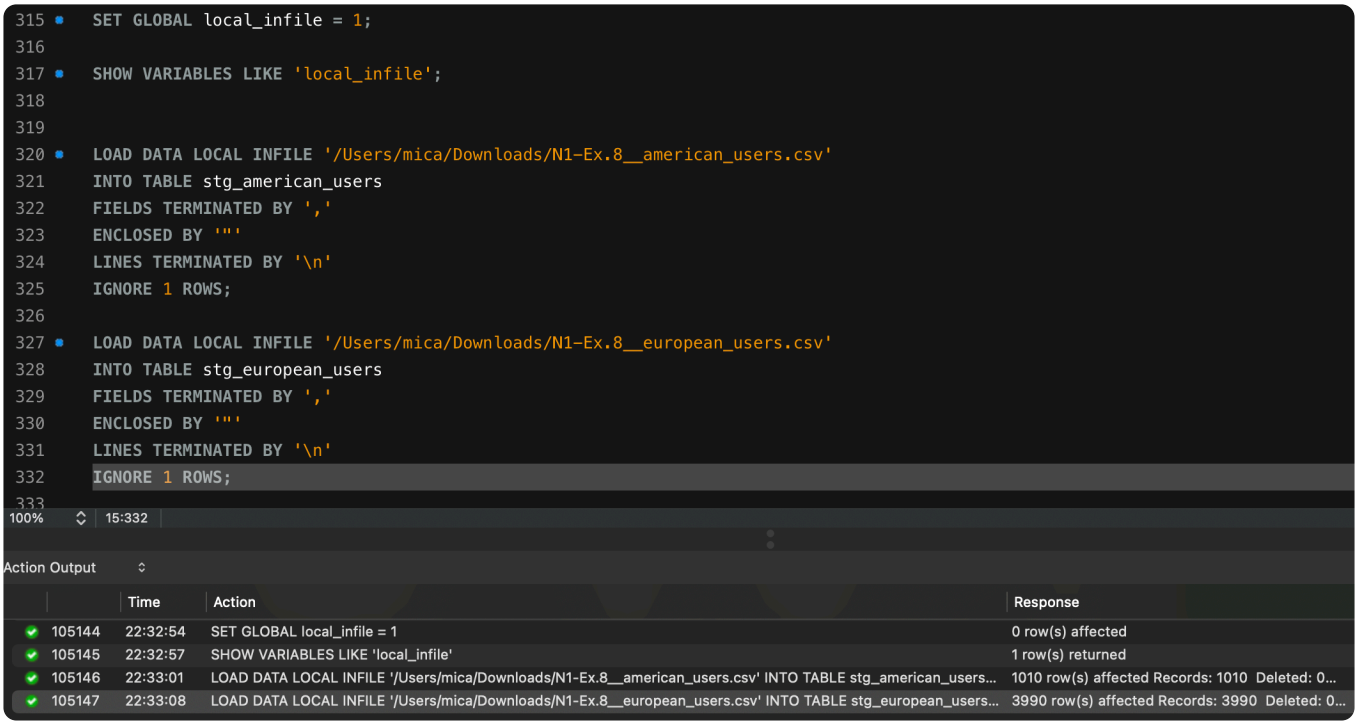
para esto, utilizamos primero tablas borrador para la importación de los usuarios americanos y europeos:

```sql
/* tenemos que crear tablas temporales para cada continente para des

CREATE TABLE stg_american_users (
    id INT,
    name VARCHAR(100),
    surname VARCHAR(100),
    phone VARCHAR(30),
    email VARCHAR(150),
    birth_date VARCHAR(30),
    country VARCHAR(100),
    city VARCHAR(100),
    postal_code VARCHAR(20),
    address VARCHAR(255),
    signup_date VARCHAR(30),
    user_segment VARCHAR(50),
    income_band VARCHAR(30)
);

CREATE TABLE stg_european_users (
    id INT,
    name VARCHAR(100),
    surname VARCHAR(100),
    phone VARCHAR(30),
    email VARCHAR(150),
    birth_date VARCHAR(30),
    country VARCHAR(100),
    city VARCHAR(100),
    postal_code VARCHAR(20),
    address VARCHAR(255),
    signup_date VARCHAR(30),
    user_segment VARCHAR(50),
    income_band VARCHAR(30)
);
```

Cargamos los datos:



```
SET GLOBAL local_infile = 1;

SHOW VARIABLES LIKE 'local_infile';


LOAD DATA LOCAL INFILE '/Users/mica/Downloads/N1-Ex.8__american_users.csv'
INTO TABLE stg_american_users
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;

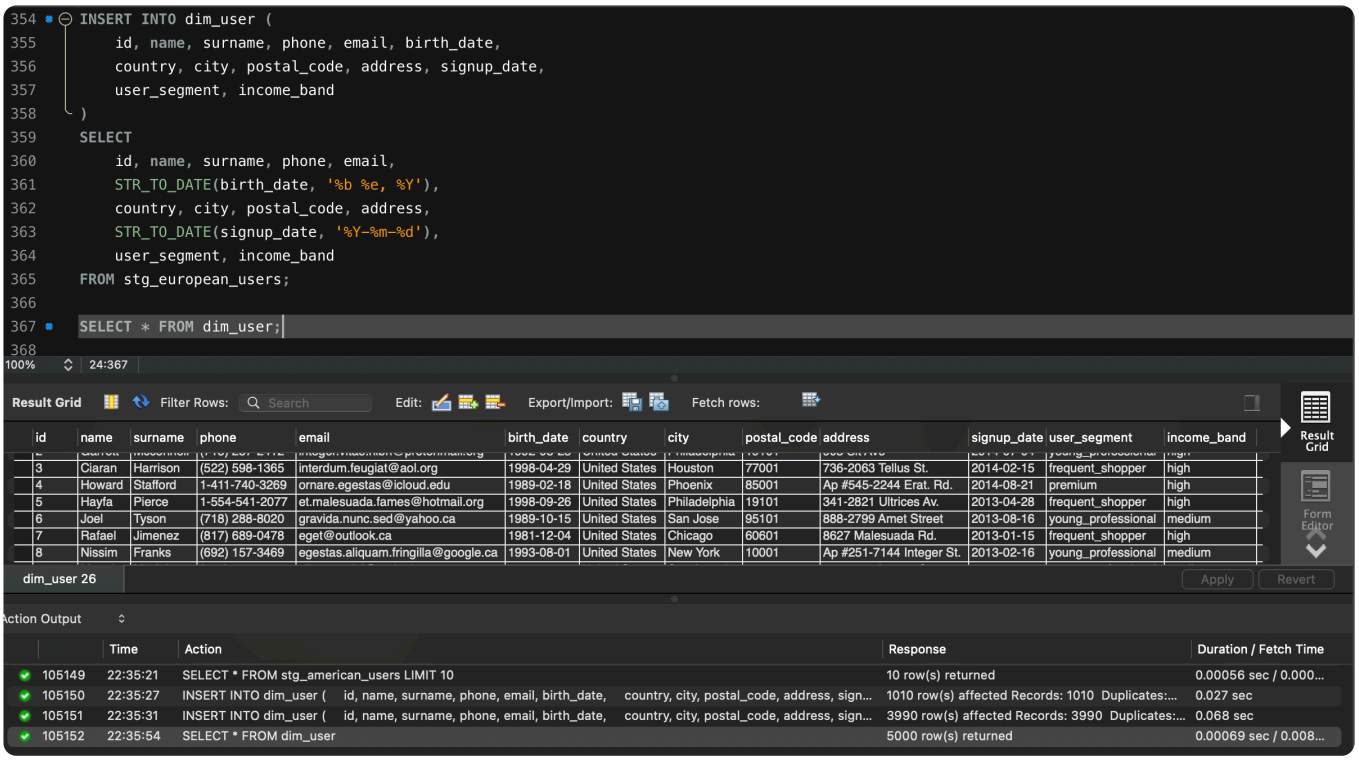
LOAD DATA LOCAL INFILE '/Users/mica/Downloads/N1-Ex.8__european_users.csv'
INTO TABLE stg_european_users
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Action Output

	Time	Action	Response
105144	22:32:54	SET GLOBAL local_infile = 1	0 row(s) affected
105145	22:32:57	SHOW VARIABLES LIKE 'local_infile'	1 row(s) returned
105146	22:33:01	LOAD DATA LOCAL INFILE '/Users/mica/Downloads/N1-Ex.8__american_users.csv' INTO TABLE stg_american_users...	1010 row(s) affected Records: 1010 Deleted: 0...
105147	22:33:08	LOAD DATA LOCAL INFILE '/Users/mica/Downloads/N1-Ex.8__european_users.csv' INTO TABLE stg_european_users...	3990 row(s) affected Records: 3990 Deleted: 0...

Comprobamos que los usuarios americanos son de un total de 1010 y los europeos de 3990.

Despues procedemos a insertar ambas tablas en *dim_user*, por ejemplo la de Europa a continuación:



```
INSERT INTO dim_user (
    id, name, surname, phone, email, birth_date,
    country, city, postal_code, address, signup_date,
    user_segment, income_band
)
SELECT
    id, name, surname, phone, email,
    STR_TO_DATE(birth_date, '%b %e, %Y'),
    country, city, postal_code, address,
    STR_TO_DATE(signup_date, '%Y-%m-%d'),
    user_segment, income_band
FROM stg_european_users;

SELECT * FROM dim_user;
```

Result Grid

id	name	surname	phone	email	birth_date	country	city	postal_code	address	signup_date	user_segment	income_band
3	Ciaran	Harrison	(522) 598-1365	interdum.feugiat@aol.org	1998-04-29	United States	Houston	77001	736-2063 Tellus St.	2014-02-15	frequent_shopper	high
4	Howard	Stafford	1-411-740-3269	ornare.egestas@icloud.edu	1989-02-18	United States	Phoenix	85001	Ap #545-2244 Erat. Rd.	2014-08-21	premium	high
5	Hayfa	Pierce	1-554-541-2077	et.malesuada.fames@hotmail.org	1998-09-26	United States	Philadelphia	19101	341-2821 Ultrices Av.	2013-04-28	frequent_shopper	high
6	Joel	Tyson	(718) 288-8020	gravida.nunc.sed@yahoo.ca	1989-10-15	United States	San Jose	95101	888-2799 Amet Street	2013-08-16	young_professional	medium
7	Rafael	Jimenez	(817) 689-0478	eget@outlook.ca	1981-12-04	United States	Chicago	60601	8627 Malesuada Rd.	2013-01-15	frequent_shopper	high
8	Nissim	Franks	(692) 157-3469	egestas.aliquam.fringilla@google.ca	1993-08-01	United States	New York	10001	Ap #251-7144 Integer St.	2013-02-16	young_professional	medium

dim_user 26

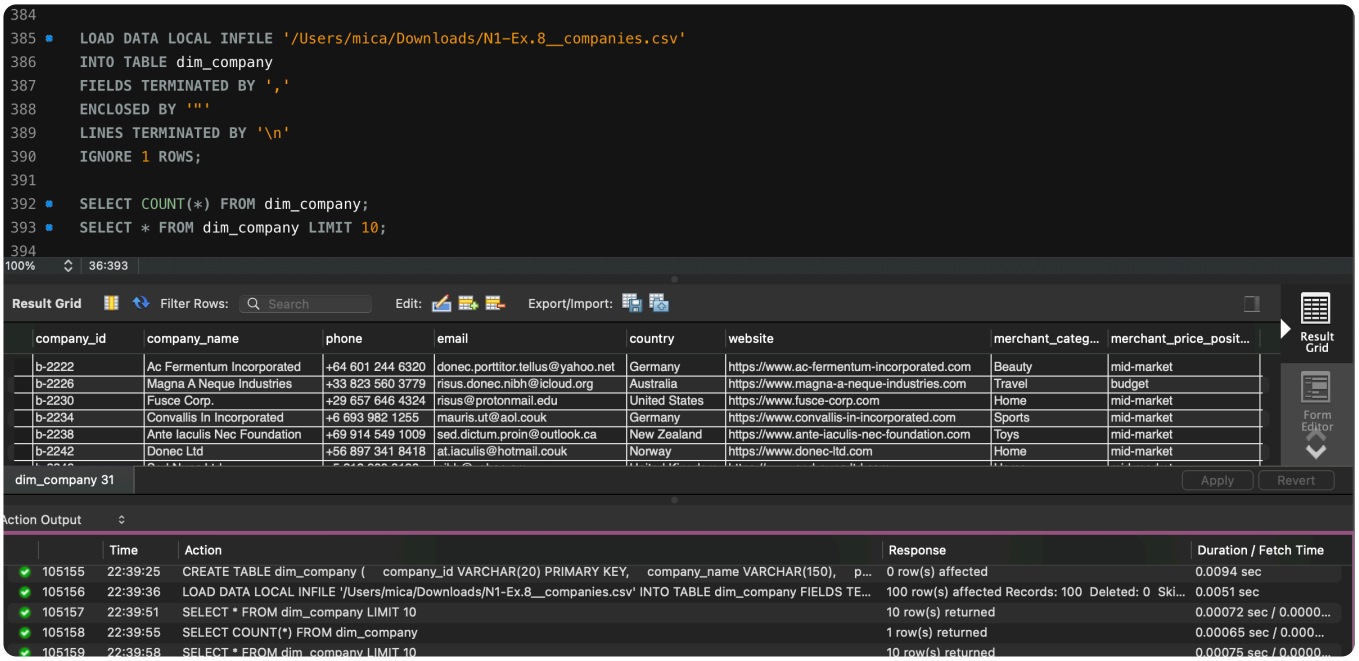
Action Output

	Time	Action	Response	Duration / Fetch Time
105149	22:35:21	SELECT * FROM stg_american_users LIMIT 10	10 row(s) returned	0.00056 sec / 0.000...
105150	22:35:27	INSERT INTO dim_user (id, name, surname, phone, email, birth_date, country, city, postal_code, address, sign...	1010 row(s) affected Records: 1010 Duplicates:...	0.027 sec
105151	22:35:31	INSERT INTO dim_user (id, name, surname, phone, email, birth_date, country, city, postal_code, address, sign...	3990 row(s) affected Records: 3990 Duplicates:...	0.068 sec
105152	22:35:54	SELECT * FROM dim_user	5000 row(s) returned	0.00069 sec / 0.008...

Vemos 5000 usuarios en total con sus diferentes datos, localizaciones, vías de contacto y capacidad de compra.

Procedemos con la creación de las tablas de 100 empresas, *dim_company*:

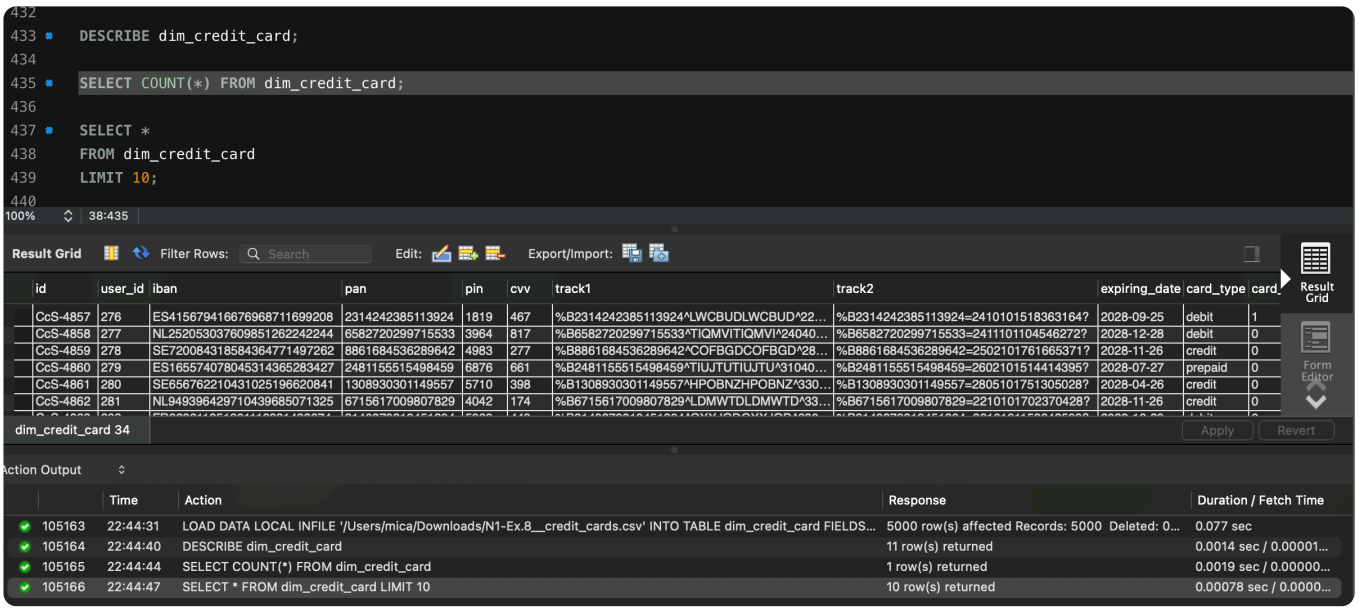
```sql
CREATE TABLE dim_company (
    company_id VARCHAR(20) PRIMARY KEY,
    company_name VARCHAR(150),
    phone VARCHAR(30),
    email VARCHAR(150),
    country VARCHAR(100),
    website VARCHAR(255),
    merchant_category VARCHAR(100),
    merchant_price_position VARCHAR(50)
);
```



Ahora la dimensión de metodo de pago: *dim_credit_card*:

```sql
CREATE TABLE dim_credit_card (
    id VARCHAR(20) PRIMARY KEY,
    user_id INT,
    iban VARCHAR(40),
    pan VARCHAR(30),
    pin VARCHAR(10),
    cvv VARCHAR(10),
    track1 TEXT,
    track2 TEXT,
    expiring_date DATE,
    card_type VARCHAR(50),
    card_renewal_flag TINYINT
);
```

Observamos 5000 observaciones de estas:

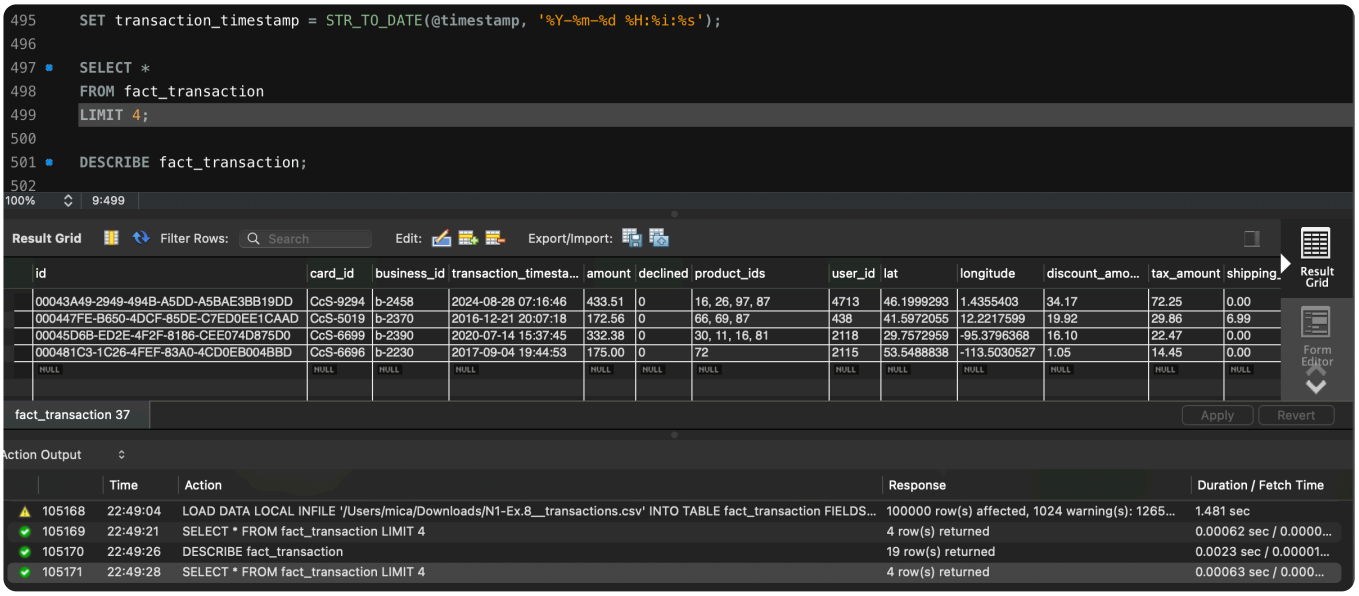


La siguiente tabla ser fundamental para relacionar las demás tablas, ya que es la base de todas las foreign keys; la tabla *fact_transaction* : Dada la cantidad de datos y para garantizar una visualización y búsqueda rápida, creamos indices de las columnas que serian mas usadas en este tipo de relaciones de datos.

```
CREATE TABLE fact_transaction (
    id CHAR(36) PRIMARY KEY,
    card_id VARCHAR(20),
    business_id VARCHAR(20),
    transaction_timestamp DATETIME,
    amount DECIMAL(12,2),
    declined TINYINT,
    product_ids TEXT,
    user_id INT,
    lat DECIMAL(10,7),
    longitude DECIMAL(10,7),
    discount_amount DECIMAL(12,2),
    tax_amount DECIMAL(12,2),
    shipping_amount DECIMAL(12,2),
    channel VARCHAR(50),
    campaign_id VARCHAR(100),
    device_type VARCHAR(50),
    is_international BOOLEAN,
    decline_reason VARCHAR(255),
    distance_km DECIMAL(12,2),

    INDEX idx_card_id (card_id),
    INDEX idx_business_id (business_id),
    INDEX idx_user_id (user_id)
);
```

Recibimos warnings a la hora de cargar los datos de transacciones, pero al comprobar, vemos que todo esta correcto:



Una vez creadas las tablas fundamentales de nuestros datos de usuarios, transacciones, tarjetas de crédito y empresas, procedemos a crear las relaciones correspondientes:

```
#entre tarjetas y usuarios

ALTER TABLE dim_credit_card
ADD CONSTRAINT fk_credit_card_user
FOREIGN KEY (user_id) REFERENCES dim_user(id);

#entre transacciones y las tarjetas

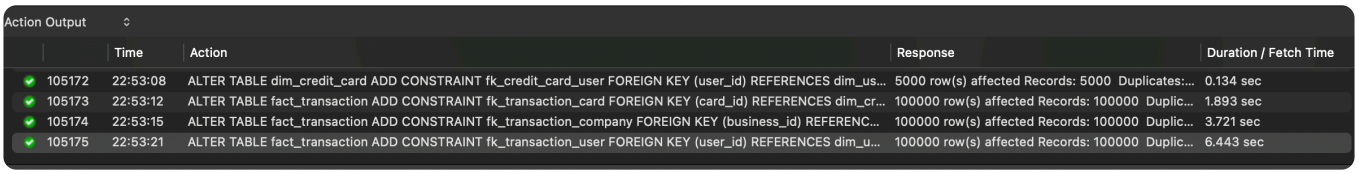
ALTER TABLE fact_transaction
ADD CONSTRAINT fk_transaction_card
FOREIGN KEY (card_id) REFERENCES dim_credit_card(id);

#entre transacciones y empresas

ALTER TABLE fact_transaction
ADD CONSTRAINT fk_transaction_company
FOREIGN KEY (business_id) REFERENCES dim_company(company_id);

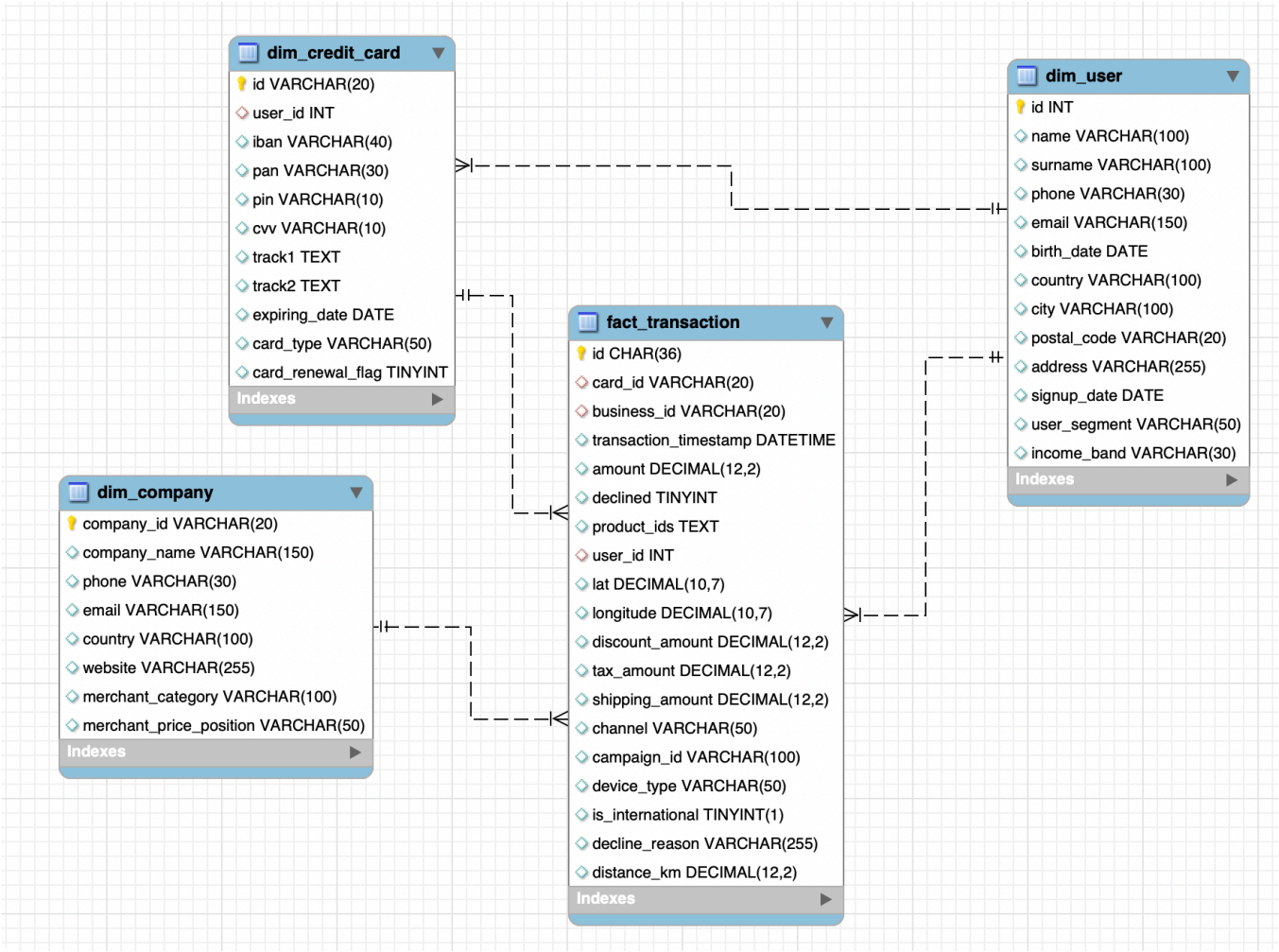
#entre transacciones y usuarios:

ALTER TABLE fact_transaction
ADD CONSTRAINT fk_transaction_user
FOREIGN KEY (user_id) REFERENCES dim_user(id);
```



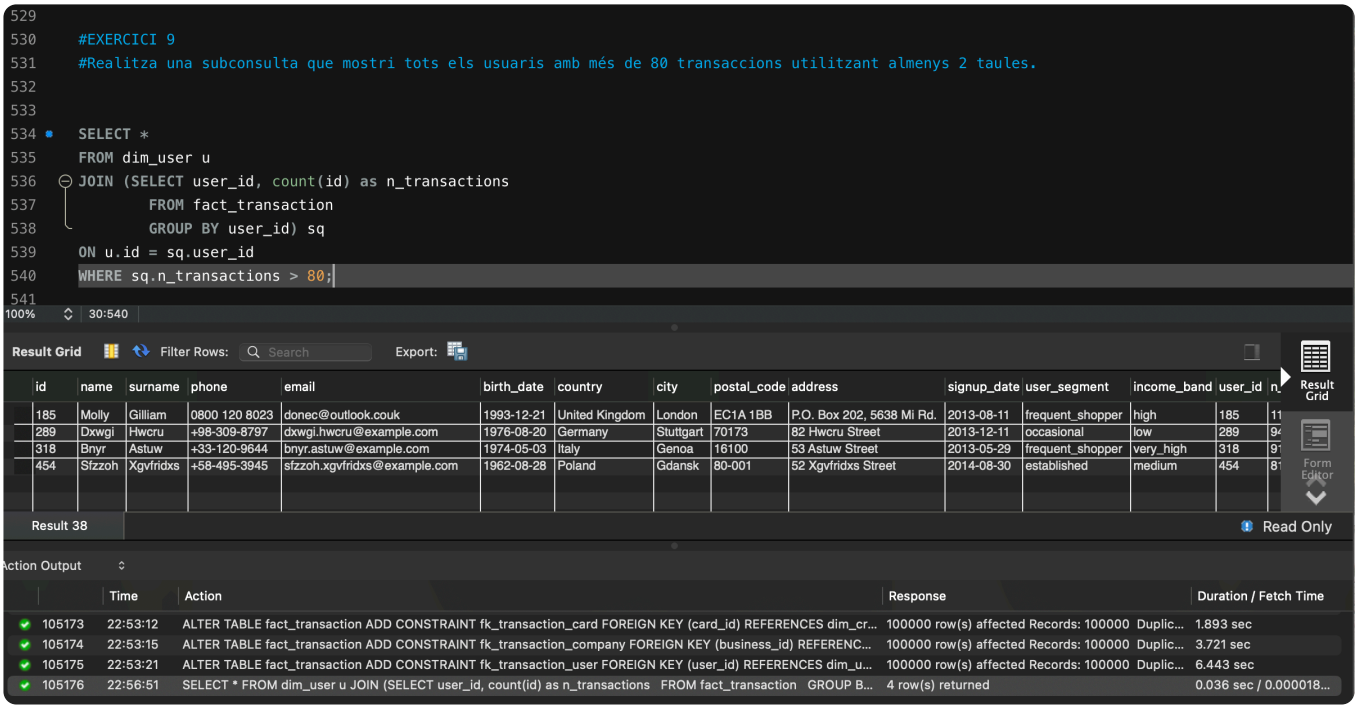
Nuestro diagrama es un modelo de estrella, siendo la tabla de hechos la dimensión de

transacciones:



Ejercicio 9

Se utiliza una subconsulta que cuenta las transacciones por usuario. Después se relaciona ese resultado con dim_user para mostrar los usuarios que han realizado más de 80 transacciones.



```sql
#EXERCICI 9
#Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

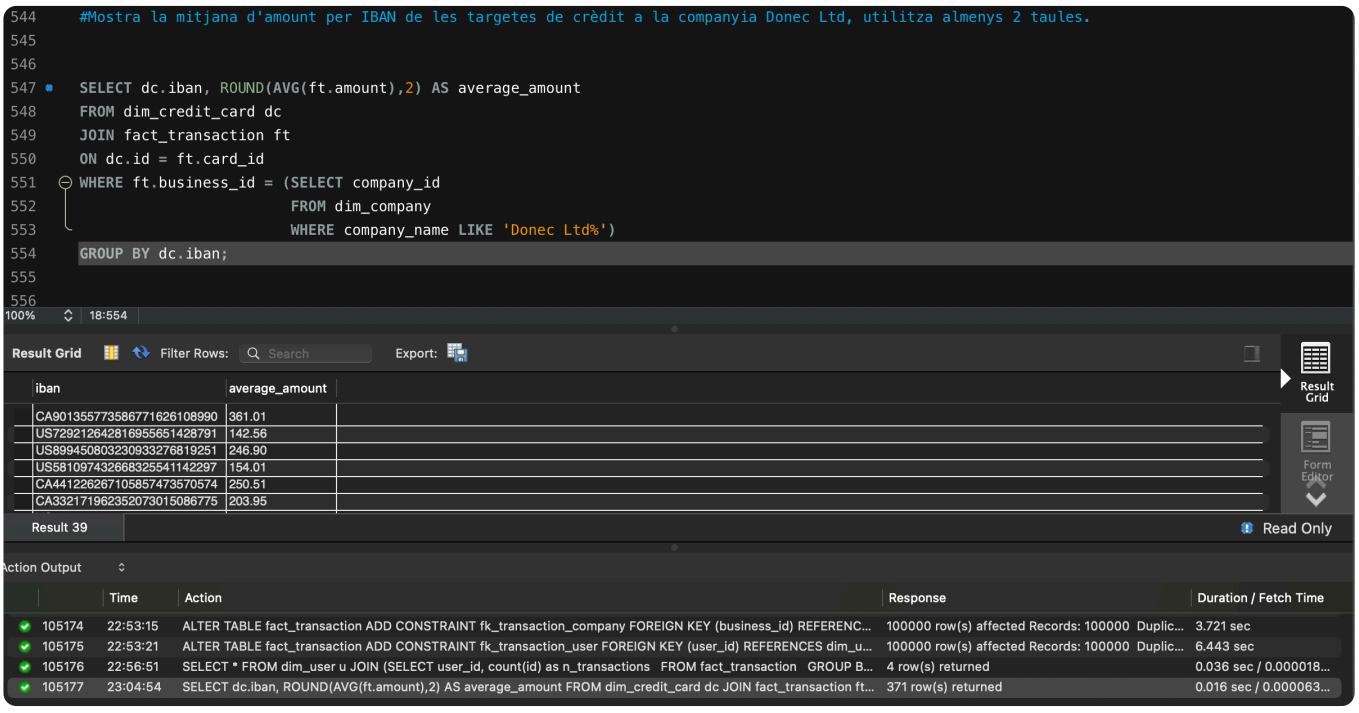

SELECT *
FROM dim_user u
JOIN (SELECT user_id, count(id) as n_transactions
        FROM fact_transaction
        GROUP BY user_id) sq
ON u.id = sq.user_id
WHERE sq.n_transactions > 80;
```

id	name	surname	phone	email	birth_date	country	city	postal_code	address	signup_date	user_segment	income_band	user_id
185	Molly	Gilliam	0800 120 8023	donec@outlook.couk	1993-12-21	United Kingdom	London	EC1A 1BB	P.O. Box 202, 5638 Mi Rd.	2013-08-11	frequent_shopper	high	185
289	Dxwgi	Hwcru	+98-309-8797	dxwgi.hwcru@example.com	1976-08-20	Germany	Stuttgart	70173	82 Hwcru Street	2013-12-11	occasional	low	289
318	Bnyr	Astuw	+33-120-9644	bnyr.astuw@example.com	1974-05-03	Italy	Genoa	16100	53 Astuw Street	2013-05-29	frequent_shopper	very_high	318
454	Sfzzoh	Xgvfridxs	+58-495-3945	sfzzoh.xgvfridxs@example.com	1962-08-28	Poland	Gdansk	80-001	52 Xgvfridxs Street	2014-08-30	established	medium	454

	Time	Action	Response	Duration / Fetch Time
105173	22:53:12	ALTER TABLE fact_transaction ADD CONSTRAINT fk_transaction_card FOREIGN KEY (card_id) REFERENCES dim_cr...	100000 row(s) affected Records: 100000 Duplic...	1.893 sec
105174	22:53:15	ALTER TABLE fact_transaction ADD CONSTRAINT fk_transaction_company FOREIGN KEY (business_id) REFERENC...	100000 row(s) affected Records: 100000 Duplic...	3.721 sec
105175	22:53:21	ALTER TABLE fact_transaction ADD CONSTRAINT fk_transaction_user FOREIGN KEY (user_id) REFERENCES dim_u...	100000 row(s) affected Records: 100000 Duplic...	6.443 sec
105176	22:56:51	SELECT * FROM dim_user u JOIN (SELECT user_id, count(id) as n_transactions FROM fact_transaction GROUP B...	4 row(s) returned	0.036 sec / 0.000018...

Ejercicio 10

Se calcula la media del importe de las transacciones por IBAN para las tarjetas que han realizado compras en la empresa Donec Ltd. Para ello se relacionan las tablas de tarjetas y transacciones y se filtra la empresa mediante una subconsulta.



```sql
#Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.


SELECT dc.iban, ROUND(AVG(ft.amount),2) AS average_amount
FROM dim_credit_card dc
JOIN fact_transaction ft
ON dc.id = ft.card_id
WHERE ft.business_id = (SELECT company_id
                        FROM dim_company
                        WHERE company_name LIKE 'Donec Ltd%')
GROUP BY dc.iban;
```

iban	average_amount
CA90135577358677162610899O	361.01
US729212642816955651428791	142.56
US899450803230933276819251	246.90
US581097432668325541142297	154.01
CA441226267105857473570574	250.51
CA332171962352073015086775	203.95

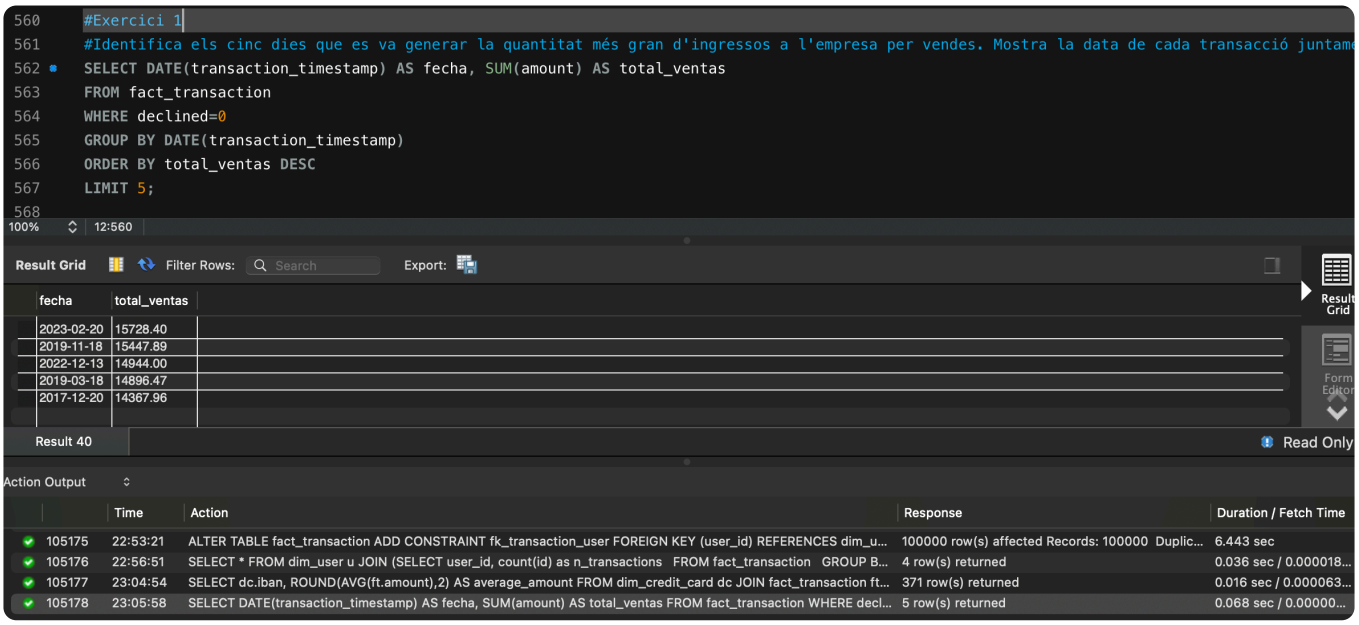
	Time	Action	Response	Duration / Fetch Time
105174	22:53:15	ALTER TABLE fact_transaction ADD CONSTRAINT fk_transaction_company FOREIGN KEY (business_id) REFERENC...	100000 row(s) affected Records: 100000 Duplic...	3.721 sec
105175	22:53:21	ALTER TABLE fact_transaction ADD CONSTRAINT fk_transaction_user FOREIGN KEY (user_id) REFERENCES dim_u...	100000 row(s) affected Records: 100000 Duplic...	6.443 sec
105176	22:56:51	SELECT * FROM dim_user u JOIN (SELECT user_id, count(id) as n_transactions FROM fact_transaction GROUP B...	4 row(s) returned	0.036 sec / 0.000018...
105177	23:04:54	SELECT dc.iban, ROUND(AVG(ft.amount),2) AS average_amount FROM dim_credit_card dc JOIN fact_transaction ft...	371 row(s) returned	0.016 sec / 0.000063...

SQL – Nivel 2

Segundo bloque de ejercicios prácticos de SQL, centrado en el trabajo con el segundo conjunto de datos.

Ejercicio 1

Se agrupan las transacciones por día y se suman sus importes para obtener los cinco días con mayores ingresos. El resultado se ordena de mayor a menor total de ventas.



```
#Exercici 1
#Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes. Mostra la data de cada transacció juntame
SELECT DATE(transaction_timestamp) AS fecha, SUM(amount) AS total_ventas
FROM fact_transaction
WHERE declined=0
GROUP BY DATE(transaction_timestamp)
ORDER BY total_ventas DESC
LIMIT 5;
```

Result Grid

fecha	total_ventas
2023-02-20	15728.40
2019-11-18	15447.89
2022-12-13	14944.00
2019-03-18	14896.47
2017-12-20	14367.96

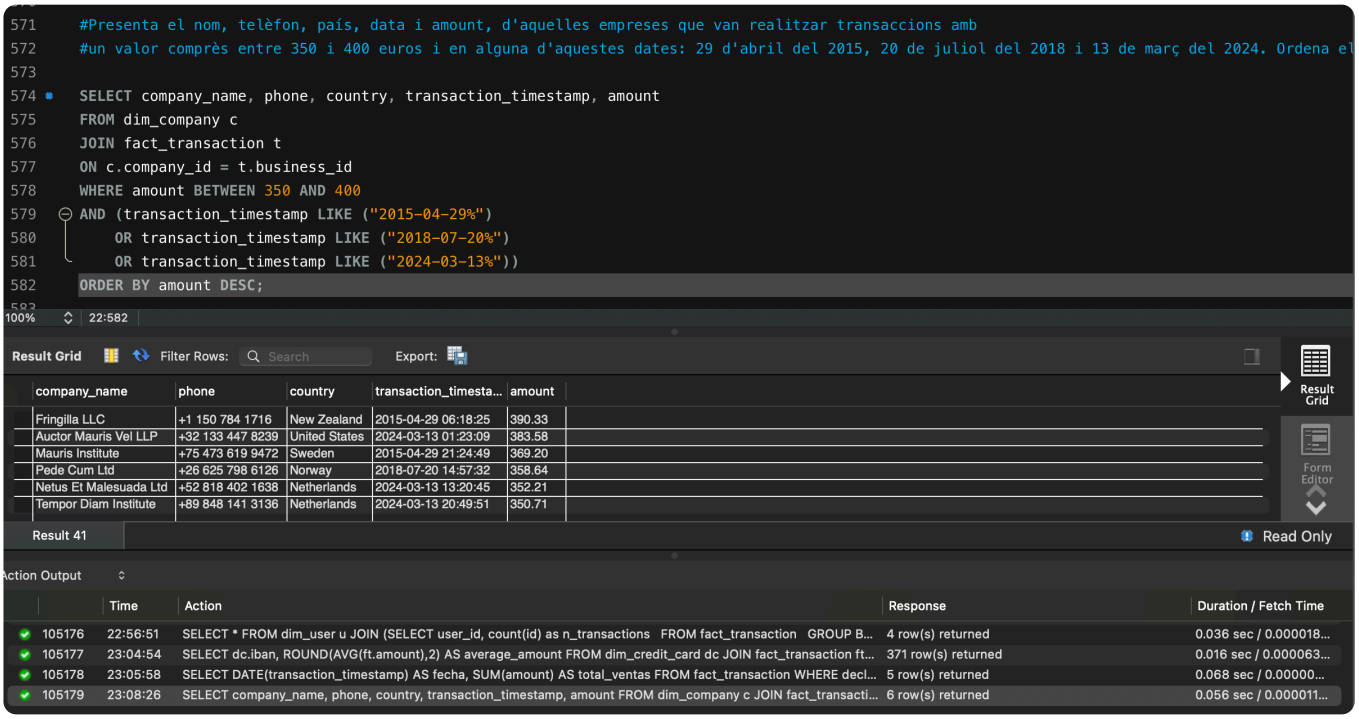
Result 40 — Read Only

Action Output

	Time	Action	Response	Duration / Fetch Time
105175	22:53:21	ALTER TABLE fact_transaction ADD CONSTRAINT fk_transaction_user FOREIGN KEY (user_id) REFERENCES dim_u...	100000 row(s) affected Records: 100000 Duplic...	6.443 sec
105176	22:56:51	SELECT * FROM dim_user u JOIN (SELECT user_id, count(id) as n_transactions FROM fact_transaction GROUP B...	4 row(s) returned	0.036 sec / 0.000018...
105177	23:04:54	SELECT dc.iban, ROUND(AVG(ft.amount),2) AS average_amount FROM dim_credit_card dc JOIN fact_transaction ft...	371 row(s) returned	0.016 sec / 0.000063...
105178	23:05:58	SELECT DATE(transaction_timestamp) AS fecha, SUM(amount) AS total_ventas FROM fact_transaction WHERE decl...	5 row(s) returned	0.068 sec / 0.00000...

Ejercicio 2

Se muestran los datos de las empresas que realizaron transacciones de entre 350 y 400 euros en cualquiera de las tres fechas indicadas. Se combinan las condiciones de importe y fecha y se ordenan los resultados por importe descendente.



```
#Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb
#un valor comprès entre 350 i 400 euros i en alguna d'aquestes dates: 29 d'abril del 2015, 20 de juliol del 2018 i 13 de març del 2024. Ordena el
SELECT company_name, phone, country, transaction_timestamp, amount
FROM dim_company c
JOIN fact_transaction t
ON c.company_id = t.business_id
WHERE amount BETWEEN 350 AND 400
AND (transaction_timestamp LIKE ("2015-04-29%")
    OR transaction_timestamp LIKE ("2018-07-20%")
    OR transaction_timestamp LIKE ("2024-03-13%"))
ORDER BY amount DESC;
```

Result Grid

company_name	phone	country	transaction_timesta...	amount
Fringilla LLC	+1 150 784 1716	New Zealand	2015-04-29 06:18:25	390.33
Auctor Mauris Vel LLP	+32 133 447 8239	United States	2024-03-13 01:23:09	383.58
Mauris Institute	+75 473 619 9472	Sweden	2015-04-29 21:24:49	369.20
Pede Cum Ltd	+26 625 798 6126	Norway	2018-07-20 14:57:32	358.64
Netus Et Malesuada Ltd	+52 818 402 1638	Netherlands	2024-03-13 13:20:45	352.21
Tempor Diam Institute	+89 848 141 3136	Netherlands	2024-03-13 20:49:51	350.71

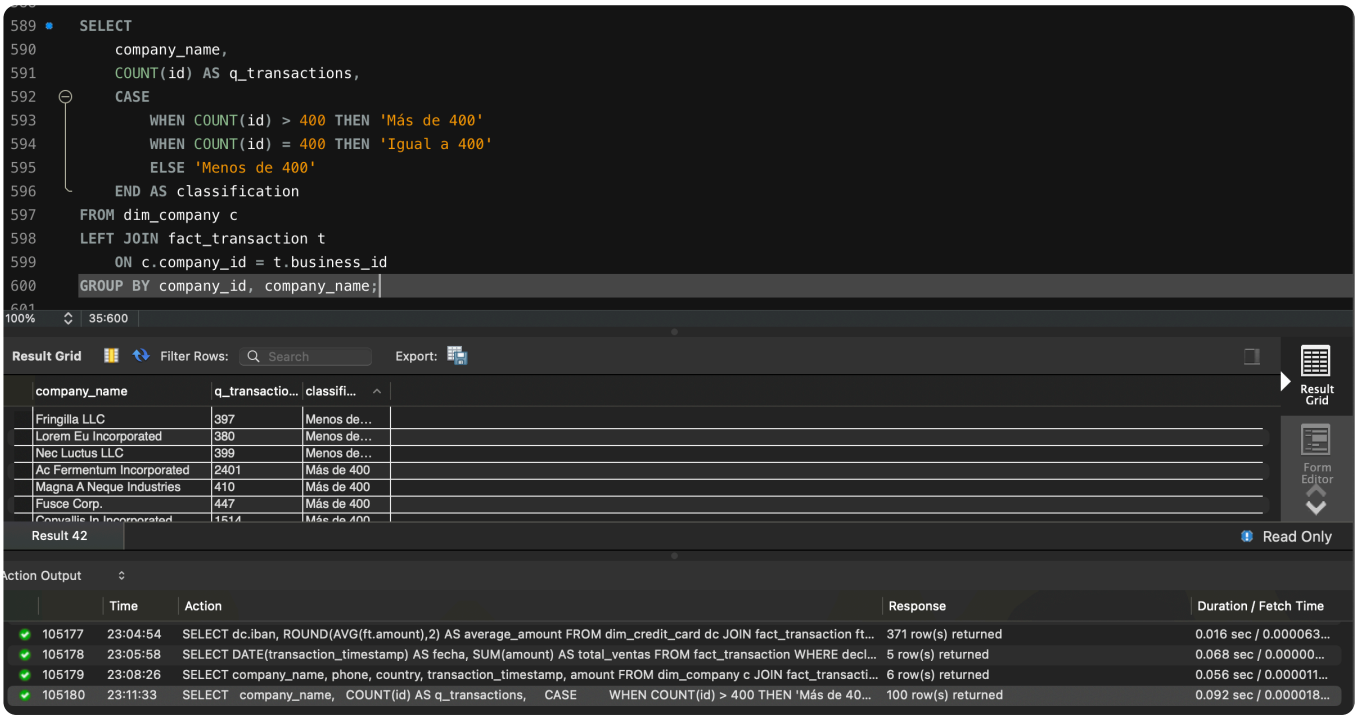
Result 41 — Read Only

Action Output

	Time	Action	Response	Duration / Fetch Time
105176	22:56:51	SELECT * FROM dim_user u JOIN (SELECT user_id, count(id) as n_transactions FROM fact_transaction GROUP B...	4 row(s) returned	0.036 sec / 0.000018...
105177	23:04:54	SELECT dc.iban, ROUND(AVG(ft.amount),2) AS average_amount FROM dim_credit_card dc JOIN fact_transaction ft...	371 row(s) returned	0.016 sec / 0.000063...
105178	23:05:58	SELECT DATE(transaction_timestamp) AS fecha, SUM(amount) AS total_ventas FROM fact_transaction WHERE decl...	5 row(s) returned	0.068 sec / 0.00000...
105179	23:08:26	SELECT company_name, phone, country, transaction_timestamp, amount FROM dim_company c JOIN fact_transacti...	6 row(s) returned	0.056 sec / 0.000011...

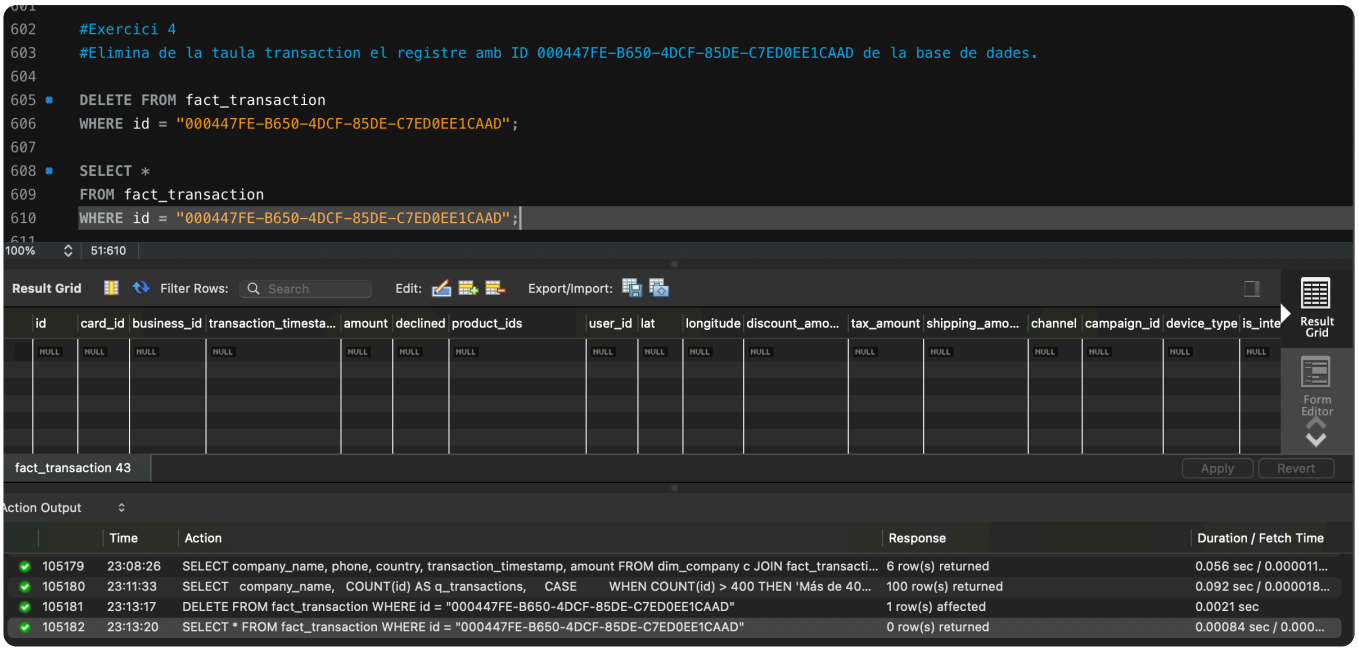
Ejercicio 3

Se cuenta el número de transacciones de cada empresa y se clasifica cada una según alcance o no las 400 operaciones. Para incluir también empresas sin transacciones se utiliza LEFT JOIN.



Ejercicio 4

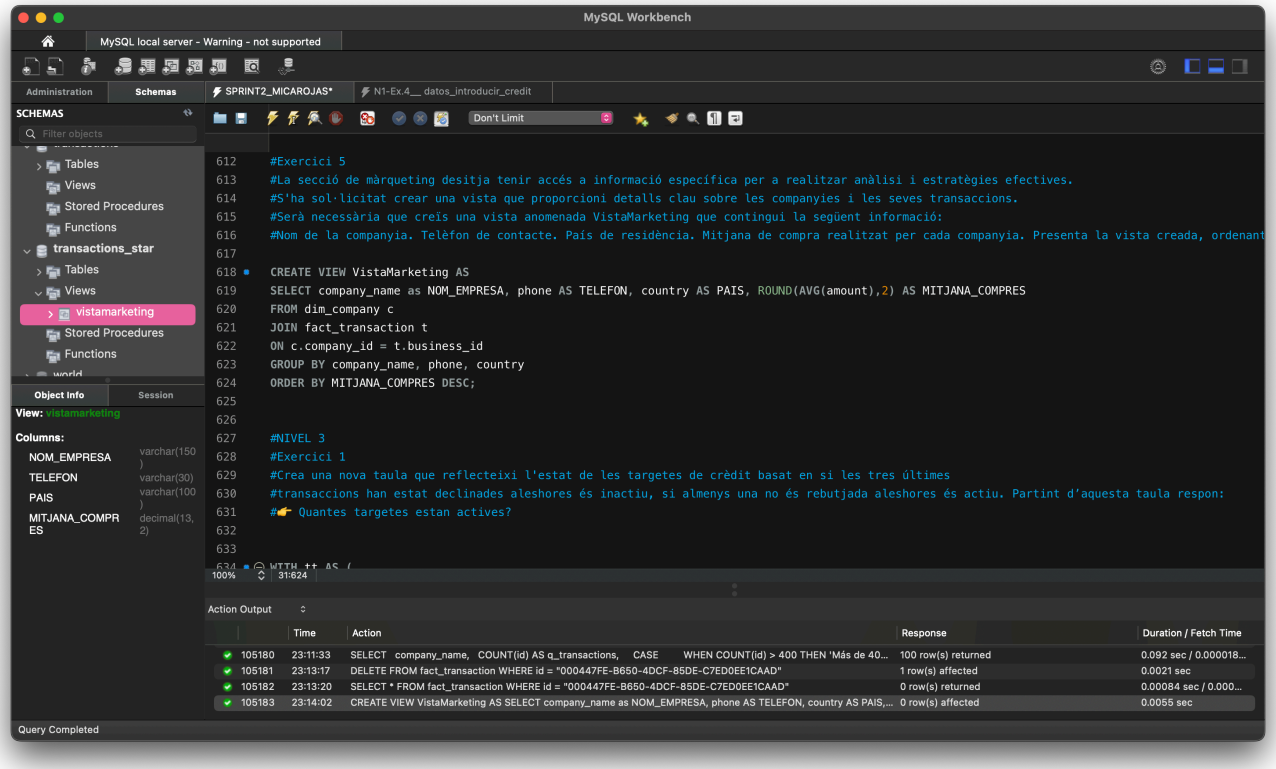
Se elimina de fact_transaction únicamente el registro cuyo identificador coincide con el indicado en el enunciado. La condición WHERE limita la operación al registro solicitado.



Ejercicio 5

Se crea la vista VistaMarketing con el nombre, teléfono y país de cada empresa, junto con la media del importe de sus transacciones. Los resultados se agrupan por empresa

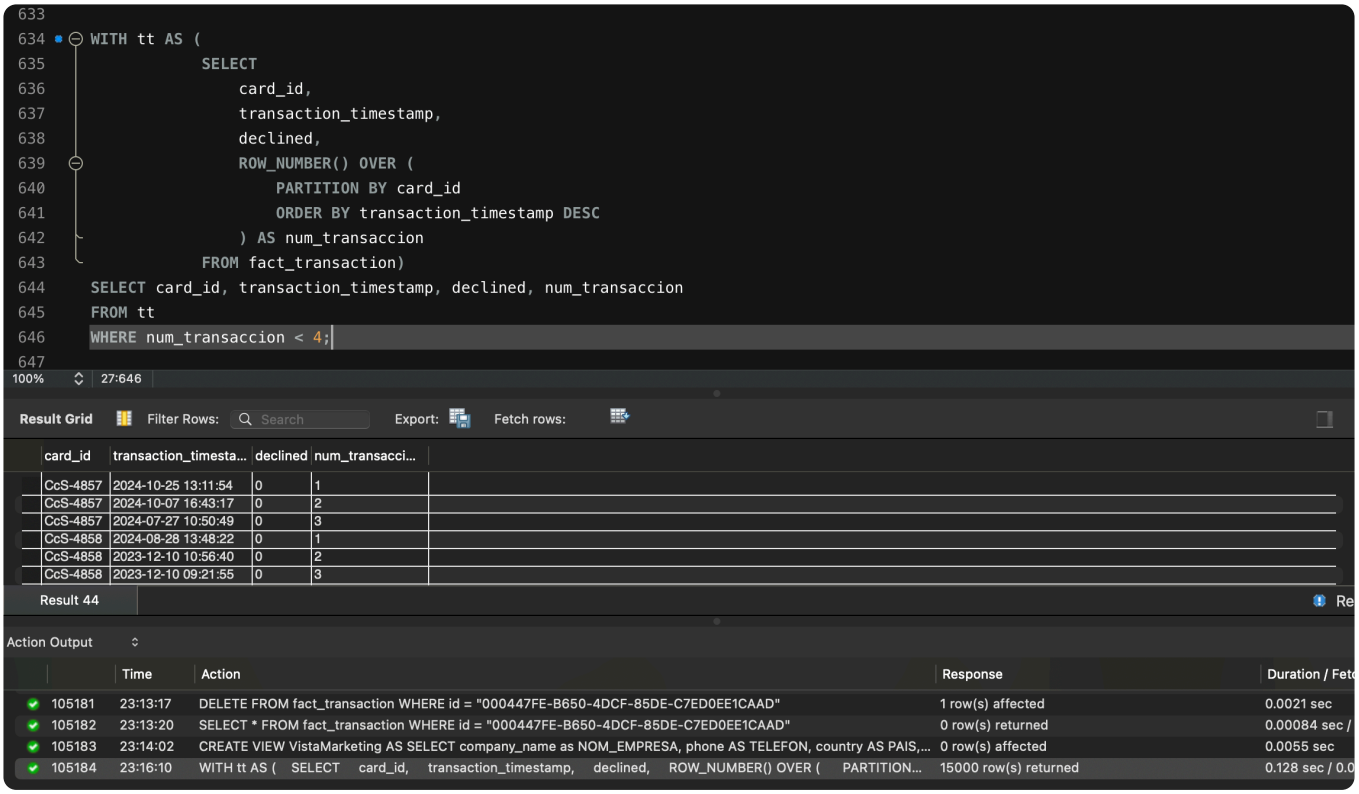
y se ordenan por la media de compra.



SQL – Nivel 3

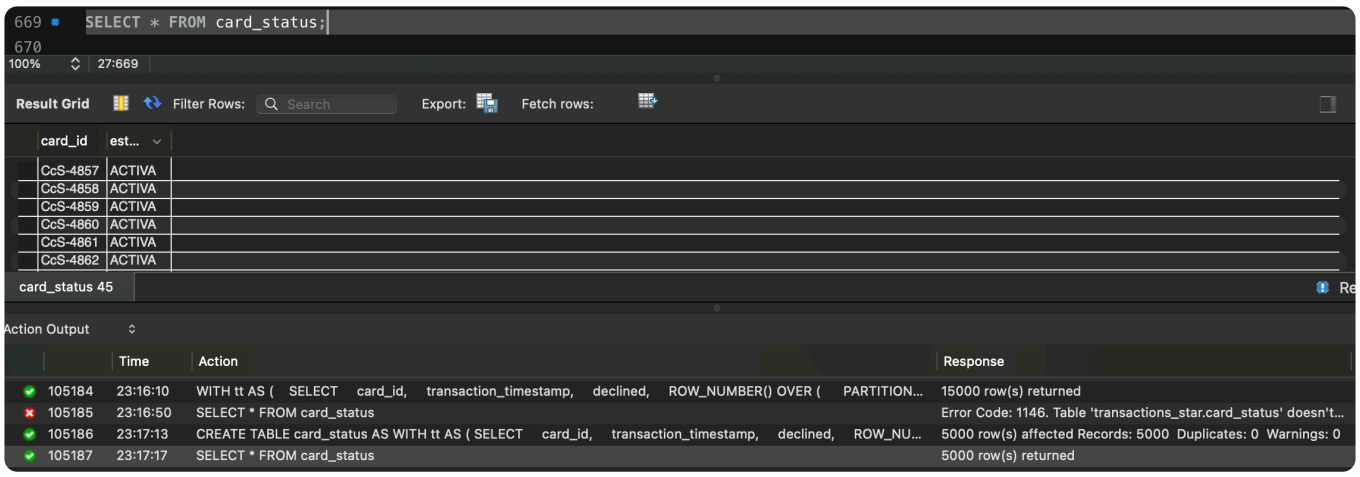
Ejercicio 1

Se determina el estado de cada tarjeta a partir de sus tres transacciones más recientes. ROW_NUMBER() ordena las operaciones por tarjeta y fecha; después se comprueba cuántas fueron rechazadas y se asigna el estado ACTIVA o INACTIVA. El resultado se guarda en una tabla.



creamos la tabla de estado de tarjeta:

```
CREATE TABLE card_status AS
WITH tt AS (
SELECT
    card_id,
    transaction_timestamp,
    declined,
    ROW_NUMBER() OVER (
        PARTITION BY card_id
        ORDER BY transaction_timestamp DESC
    ) AS num_transaccion
FROM fact_transaction)
SELECT
        card_id,
    CASE
                WHEN SUM(declined) = 3 THEN 'INACTIVA'
                ELSE 'ACTIVA'
        END AS estado
FROM tt
WHERE num_transaccion < 4
GROUP BY card_id;
```

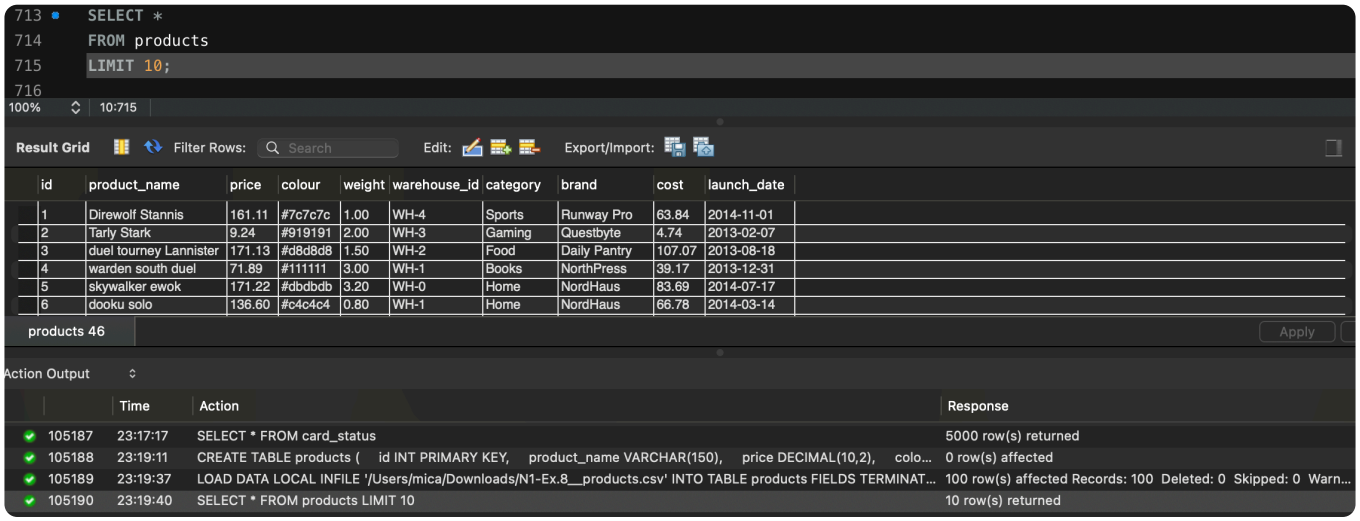


Vemos que todas las tarjetas están activadas, las 5000!

Ejercicio 2

Se separan los identificadores incluidos en product_ids para obtener una fila por producto y transacción. Después se relacionan los datos con la tabla products mediante una tabla intermedia y se cuenta cuántas veces aparece vendido cada producto.

```
CREATE TABLE products (
    id INT PRIMARY KEY,
    product_name VARCHAR(150),
    price DECIMAL(10,2),
    colour VARCHAR(20),
    weight DECIMAL(10,2),
    warehouse_id VARCHAR(20),
    category VARCHAR(100),
    brand VARCHAR(100),
    cost DECIMAL(10,2),
    launch_date DATE
);
```



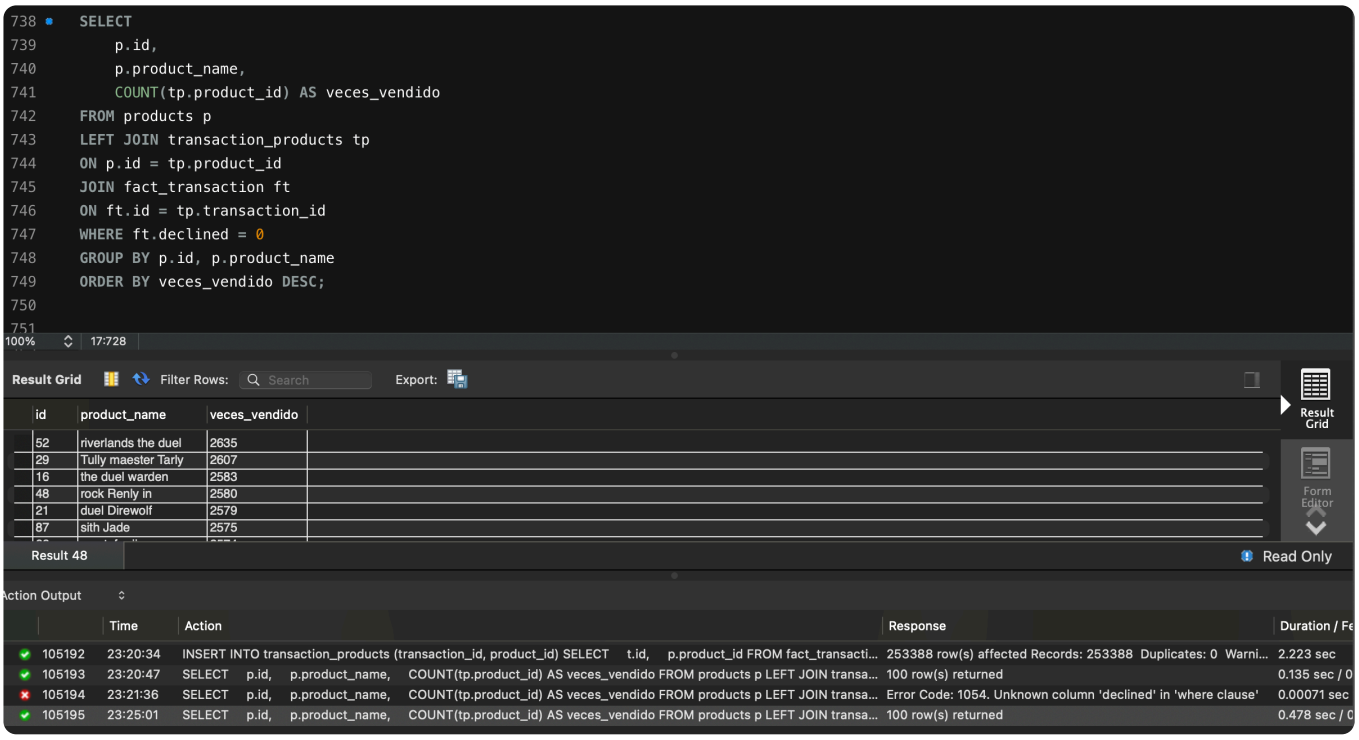
creación de tabla intermedia:

```sql
CREATE TABLE transaction_products (
    transaction_id CHAR(36),
    product_id INT,
    PRIMARY KEY (transaction_id, product_id),
    FOREIGN KEY (transaction_id) REFERENCES fact_transaction(id),
    FOREIGN KEY (product_id) REFERENCES products(id)
);
```

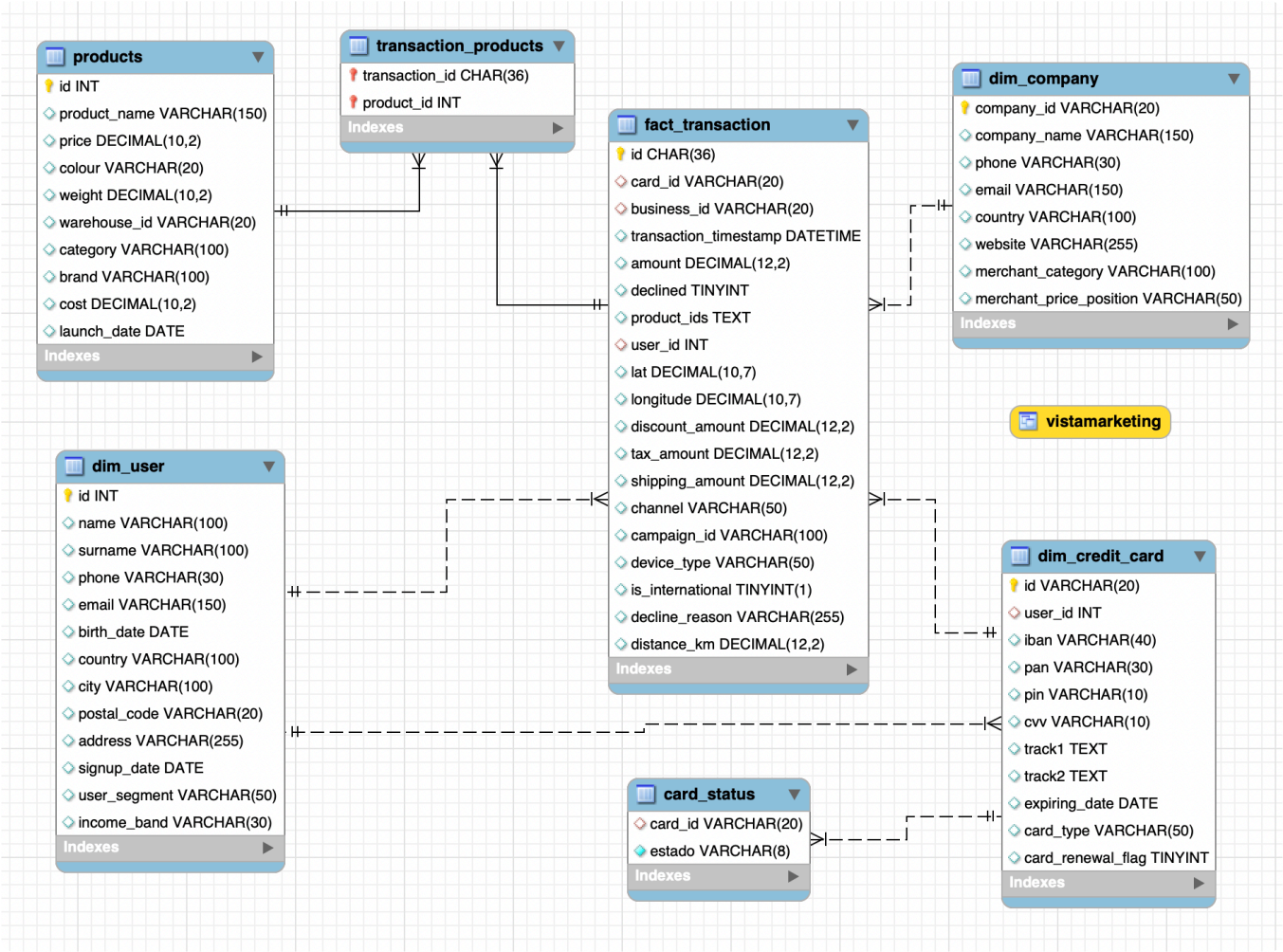
Hemos de separar la columna de productos, ya que los productos se encuentran todos juntos en formato de Texto, para eso usamos JSON;

```sql
INSERT INTO transaction_products (transaction_id, product_id)
SELECT
    t.id,
    p.product_id
FROM fact_transaction t,
JSON_TABLE(
    CONCAT('[', t.product_ids, ']'),
    '$[*]' COLUMNS (
        product_id INT PATH '$'
    )
) AS p;
```

Usamos las tablas creadas e intermedias para observar que el producto mas vendido es el de la primera fila:



El resultado final de relaciones del database *transaction_star* es el siguiente:


products
id INT
product_name VARCHAR(150)
price DECIMAL(10,2)
colour VARCHAR(20)
weight DECIMAL(10,2)
warehouse_id VARCHAR(20)
category VARCHAR(100)
brand VARCHAR(100)
cost DECIMAL(10,2)
launch_date DATE
Indexes
transaction_products
transaction_id CHAR(36)
product_id INT
Indexes
fact_transaction
id CHAR(36)
card_id VARCHAR(20)
business_id VARCHAR(20)
transaction_timestamp DATETIME
amount DECIMAL(12,2)
declined TINYINT
product_ids TEXT
user_id INT
lat DECIMAL(10,7)
longitude DECIMAL(10,7)
discount_amount DECIMAL(12,2)
tax_amount DECIMAL(12,2)
shipping_amount DECIMAL(12,2)
channel VARCHAR(50)
campaign_id VARCHAR(100)
device_type VARCHAR(50)
is_international TINYINT(1)
decline_reason VARCHAR(255)
distance_km DECIMAL(12,2)
Indexes
dim_company
company_id VARCHAR(20)
company_name VARCHAR(150)
phone VARCHAR(30)
email VARCHAR(150)
country VARCHAR(100)
website VARCHAR(255)
merchant_category VARCHAR(100)
merchant_price_position VARCHAR(50)
Indexes
vistamarketing
dim_user
id INT
name VARCHAR(100)
surname VARCHAR(100)
phone VARCHAR(30)
email VARCHAR(150)
birth_date DATE
country VARCHAR(100)
city VARCHAR(100)
postal_code VARCHAR(20)
address VARCHAR(255)
signup_date DATE
user_segment VARCHAR(50)
income_band VARCHAR(30)
Indexes
dim_credit_card
id VARCHAR(20)
user_id INT
iban VARCHAR(40)
pan VARCHAR(30)
pin VARCHAR(10)
cvv VARCHAR(10)
track1 TEXT
track2 TEXT
expiring_date DATE
card_type VARCHAR(50)
card_renewal_flag TINYINT
Indexes
card_status
card_id VARCHAR(20)
estado VARCHAR(8)
Indexes